

Tractable Language Model Programming: Themes and Prospects

Kyle Richardson

Allen Institute for AI (AI2)

August 2026

The 9th Workshop on Tractable Probabilistic Modeling (TPM@UAI)



General purpose large language models (LLMs)



Prompt

User

Briefly tell me in a sentence about the workshop on tractable probabilistic modeling (TPM). If you were to give a talk, what might you talk about?

40t

Assistant

Response



ChatGPT

Assistant

The Workshop on Tractable Probabilistic Modeling focuses on probabilistic models that support efficient exact or approximate inference, such as probabilistic circuits, sum-product networks, and related neural-symbolic methods. If I gave a talk, I'd discuss how tractable probabilistic models can be combined with modern deep generative models to provide calibrated uncertainty, interpretable structure, and efficient reasoning.

General purpose large language models (LLMs)



Prompt

User

Briefly tell me in a sentence about the workshop on tractable probabilistic modeling (TPM). If you were to give a talk, what might you talk about?

40t

Assistant

Response



ChatGPT

Assistant

The Workshop on Tractable Probabilistic Modeling focuses on probabilistic models that support efficient exact or approximate inference, such as probabilistic circuits, sum-product networks, and related neural-symbolic methods. If I gave a talk, I'd discuss how tractable probabilistic models can be combined with modern deep generative models to provide calibrated uncertainty, interpretable structure, and efficient reasoning.

- ▶ **General purpose models:** trained at massive scales, used *as-is* and directly for a wide range of problems.

General purpose large language models (LLMs)



Prompt

User:
Briefly tell me in a sentence about the workshop on tractable probabilistic modeling (TPM). If you were to give a talk, what might you talk about?

Response



Chat

Models have far exceeded expectations

OpenAI's GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet are the first models that support multimodal inputs and outputs. These models can process and generate text, images, audio, and video. Gemini 1.5 Pro also supports long context windows, allowing it to process and generate text across thousands of tokens. Gemini 1.5 Pro also supports structured outputs, allowing it to generate JSON, XML, and other structured data formats. Gemini 1.5 Pro also supports function calling, allowing it to interact with external systems and APIs. Gemini 1.5 Pro also supports tool use, allowing it to perform tasks such as web browsing, image search, and code execution. Gemini 1.5 Pro also supports retrieval-augmented generation (RAG), allowing it to generate text based on information retrieved from a knowledge base. Gemini 1.5 Pro also supports multi-modal reasoning, allowing it to perform tasks such as image classification, image captioning, and image question answering. Gemini 1.5 Pro also supports multi-modal generation, allowing it to generate images, audio, and video from text prompts. Gemini 1.5 Pro also supports multi-modal summarization, allowing it to generate summaries of text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal translation, allowing it to translate text, images, audio, and video between different languages. Gemini 1.5 Pro also supports multi-modal sentiment analysis, allowing it to analyze the sentiment of text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal topic classification, allowing it to classify text, images, audio, and video into different topics. Gemini 1.5 Pro also supports multi-modal entity recognition, allowing it to identify entities in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal relation extraction, allowing it to extract relationships between entities in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal event detection, allowing it to detect events in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal activity recognition, allowing it to recognize activities in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal emotion recognition, allowing it to recognize emotions in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal personality recognition, allowing it to recognize personalities in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal social network analysis, allowing it to analyze social networks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal recommendation systems, allowing it to recommend text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal fraud detection, allowing it to detect fraud in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal spam detection, allowing it to detect spam in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal phishing detection, allowing it to detect phishing in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal malware detection, allowing it to detect malware in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal ransomware detection, allowing it to detect ransomware in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal data breach detection, allowing it to detect data breaches in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal insider threat detection, allowing it to detect insider threats in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal supply chain risk management, allowing it to manage supply chain risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal cybersecurity risk management, allowing it to manage cybersecurity risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal financial risk management, allowing it to manage financial risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal operational risk management, allowing it to manage operational risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal compliance risk management, allowing it to manage compliance risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal reputational risk management, allowing it to manage reputational risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal strategic risk management, allowing it to manage strategic risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal environmental risk management, allowing it to manage environmental risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal social risk management, allowing it to manage social risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal economic risk management, allowing it to manage economic risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal political risk management, allowing it to manage political risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal legal risk management, allowing it to manage legal risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal ethical risk management, allowing it to manage ethical risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal governance risk management, allowing it to manage governance risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal sustainability risk management, allowing it to manage sustainability risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal resilience risk management, allowing it to manage resilience risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal continuity risk management, allowing it to manage continuity risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal disaster risk management, allowing it to manage disaster risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal crisis risk management, allowing it to manage crisis risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal incident risk management, allowing it to manage incident risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal threat risk management, allowing it to manage threat risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal vulnerability risk management, allowing it to manage vulnerability risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal exposure risk management, allowing it to manage exposure risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal consequence risk management, allowing it to manage consequence risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal likelihood risk management, allowing it to manage likelihood risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal severity risk management, allowing it to manage severity risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal detectability risk management, allowing it to manage detectability risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal attribution risk management, allowing it to manage attribution risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal impact risk management, allowing it to manage impact risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal response risk management, allowing it to manage response risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal recovery risk management, allowing it to manage recovery risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal mitigation risk management, allowing it to manage mitigation risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal prevention risk management, allowing it to manage prevention risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal preparedness risk management, allowing it to manage preparedness risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal response risk management, allowing it to manage response risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal recovery risk management, allowing it to manage recovery risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal mitigation risk management, allowing it to manage mitigation risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal prevention risk management, allowing it to manage prevention risks in text, images, audio, and video. Gemini 1.5 Pro also supports multi-modal preparedness risk management, allowing it to manage preparedness risks in text, images, audio, and video.

- **General purpose models:** trained at massive scales, used *as-is* and directly for a wide range of problems.

Language models as agent simulators

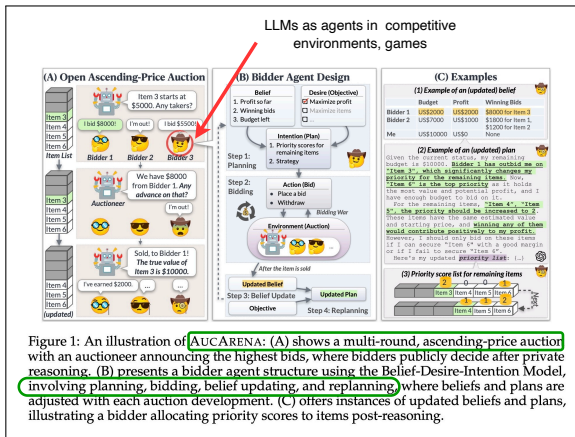


Figure 1: An illustration of AUCARENA: (A) shows a multi-round, ascending-price auction with an auctioneer announcing the highest bids, where bidders publicly decide after private reasoning. (B) presents a bidder agent structure using the Belief-Desire-Intention Model, involving planning, bidding, belief updating, and replanning, where beliefs and plans are adjusted with each auction development. (C) offers instances of updated beliefs and plans, illustrating a bidder allocating priority scores to items post-reasoning.

- Can we use LMs to simulate complex social dynamics? (Chen et al.2023; Zhang et al.2024; Yang et al.2025)

Language models as agent simulators

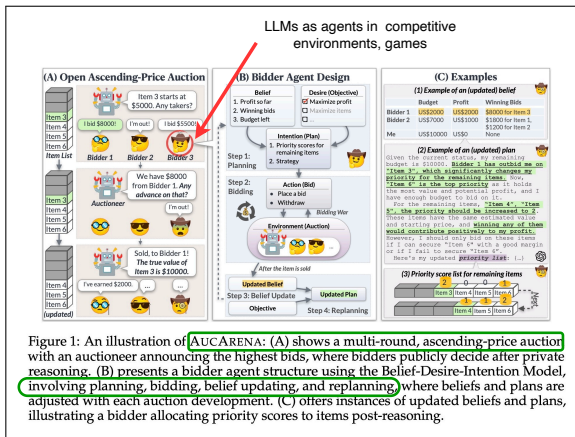


Figure 1: An illustration of AUCARENA: (A) shows a multi-round, ascending-price auction with an auctioneer announcing the highest bids, where bidders publicly decide after private reasoning. (B) presents a bidder agent structure using the Belief-Desire-Intention Model, involving planning, bidding, belief updating, and replanning, where beliefs and plans are adjusted with each auction development. (C) offers instances of updated beliefs and plans, illustrating a bidder allocating priority scores to items post-reasoning.

Valuable tool for running social science experiments, testing theories of language interaction, complex reasoning, adversarial language experts.

Language models as part of complex systems


Machine
learning
experiment



Model
generated
code



ML Experiment Execution Engine

Query: Implement an encoder-only model using transformers that can do multiple-choice QA

Running step: 1

Step 1

system thought : To implement an encoder-only model for multiple-choice question answering (QA) using the transformers library from Hugging Face, I'll first outline the steps in Python. This will involve loading a pre-trained encoder model, perhaps something like BERT or RoBERTa, and then adapting it to handle multiple-choice QA tasks. The model will receive a question and several possible answers as input and will have to select the most likely answer. I will use the `transformers` library for this purpose.

Action (execute)

```
from transformers import AutoModelForMultipleChoice, AutoTokenizer

model_name = 'bert-base-uncased'

# Load model and tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForMultipleChoice.from_pretrained(model_name)

print('Model and tokenizer loaded successfully.')
```

Observation

▼ Details

Model and tokenizer loaded successfully.

Running step: 2

Step 2

jupyter automated_ml_notebook Local Checkpoint 33 minutes ago

```
[1]: from transformers import AutoModelForMultipleChoice, AutoTokenizer

# System explanation
'''To implement an encoder-only model for multiple-choice question answering (QA) using the transformers library from HuggingFace, I'll first outline the steps in Python. This will involve loading a pre-trained encoder model, perhaps something like BERT or RoBERTa, and then adapting it to handle multiple-choice QA tasks.

Below is an example implementation to start:'''

[2]: model_name = 'bert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForMultipleChoice.from_pretrained(model_name)
print('Model and tokenizer loaded successfully.')
```

Model and tokenizer loaded successfully.

Observation

▼ Details

Model and tokenizer loaded successfully.

Experiment automation

- ▶ Automated research (Bogin et al.2024; Yu et al.2026) and general agentic benchmarking (Bragg et al.2026).

Language models as part of complex systems, agents

Language Modeling by Language Models

Junyan Cheng^{1*} Peter Clark² Kyle Richardson²

Allen Institute for AI¹ Dartmouth College²
j.c.chen@allenai.org kyle@dartmouth.edu

✦ <https://github.com/allenai/genesys>

Abstract

Can we leverage LLMs to model the process of discovering novel language model architectures? Inspired by real research, we propose a multi-agent LLM approach that simulates the conventional stages of research, from ideation and literature search (proposal stage) to design implementation (code generation), generative pre-training, and downstream evaluation (verification). Using ideas from scaling laws, our system *Genesys* employs a *Ladder of Scales* approach; new designs are proposed, adversarially reviewed, implemented, and selectively verified at increasingly larger model scales (14M~350M parameters) with a narrowing budget (the number of models we can train at each scale). To help make discovery efficient and factorizable, *Genesys* uses a novel genetic programming backbone, which we show has empirical advantages over commonly used direct prompt generation workflows (e.g., ~86% percentage point improvement in successful design generation, a key bottleneck). We report experiments involving 1,162 newly discovered designs (1,062 fully verified through pre-training) and find the best designs to be highly competitive with known architectures (e.g., outperform OPT2, Mamba2, etc., on 6/9 common benchmarks). We couple these results with comprehensive system-level ablations and formal results, which give broader insights into the design of effective autonomous LLM-driven discovery systems.

(Cheng et al.2025)

TINYSCIENTIST: An Interactive, Extensible, and Controllable Framework for Building Research Agents

Haofei Yu^{1*} Keyang Xuan^{1*} Fenghai Li^{1*}
Kunlun Zhu¹ Zijie Lei¹ Jiaxun Zhang¹ Ziheng Qi¹
Kyle Richardson² Jiaxuan You¹

¹University of Illinois Urbana-Champaign,

²Allen Institute for Artificial Intelligence

Abstract

Automatic research with Large Language Models (LLMs) is rapidly gaining importance, driving the development of increasingly complex workflows involving multi-agent systems, planning, tool usage, code execution, and human-agent interaction to accelerate research processes. However, as more researchers and developers begin to use and build upon these tools and platforms, the complexity and difficulty of extending and maintaining such agentic workflows have become a significant challenge, particularly as algorithms and architectures continue to advance. To address this growing complexity, TINYSCIENTIST identifies the essential components of the automatic research workflow and proposes an interactive, extensible, and controllable framework that easily adapts to new tools and supports iterative growth. We provide an open-source codebase¹, an interactive web demonstration², and a PyPI Python package³ to make state-of-the-art auto-research pipelines broadly accessible to every researcher and developer.

end research pipelines (Jansen et al., 2025; Lu et al., 2024; Yamada et al., 2025; Li et al., 2024b; Cheng et al., 2025). Recent advances in this area leverage methods including multi-agent collaboration (Schmidgall et al., 2025), tool using (Skarinski et al., 2024), and tree-based search (Yamada et al., 2025) to augment its performance.

In spite of this success, however, existing automatic research systems often design and use agentic frameworks that are overly complex and difficult to use and extend without significant technical expertise. These challenges stem from three key issues: (1) **lack of interactivity**: human researchers struggle to engage with the specific agent’s research progress due to the complexity of research intents and unclear communication interfaces (Zou et al., 2025; Liu et al., 2025b), making feedback incorporation challenging. (2) **limited extensibility**: existing representative frameworks rely on rigid, tool-specific designs (Zhang et al., 2025), making it hard to integrate new tools or adapt to different research domains. (3) **insufficient controllability**: many

(Yu et al.2025)

A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

Lots of optimism, hubris, Nobel prizes....

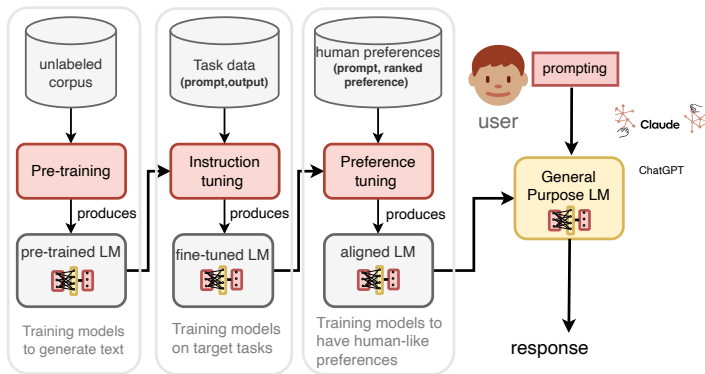
A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

Missing **semantic** and **algorithmic** foundations.

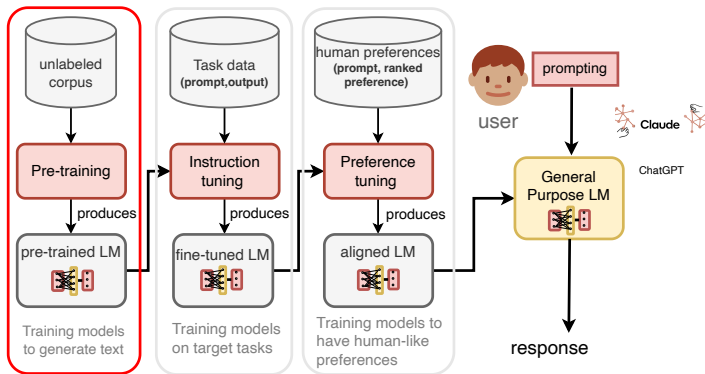
A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

Missing **semantic** and **algorithmic** foundations.

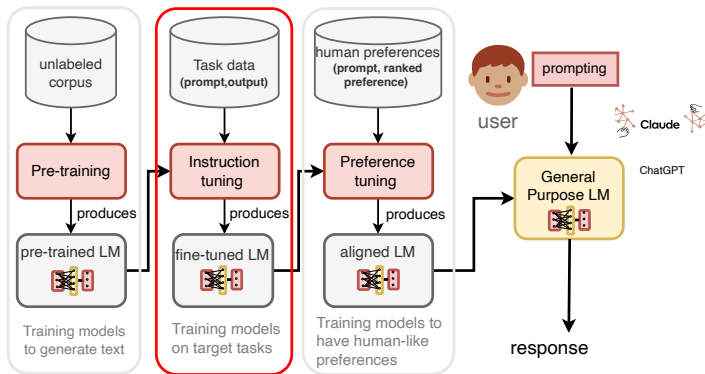
How do we get to general-purpose LLMs? **recipe**



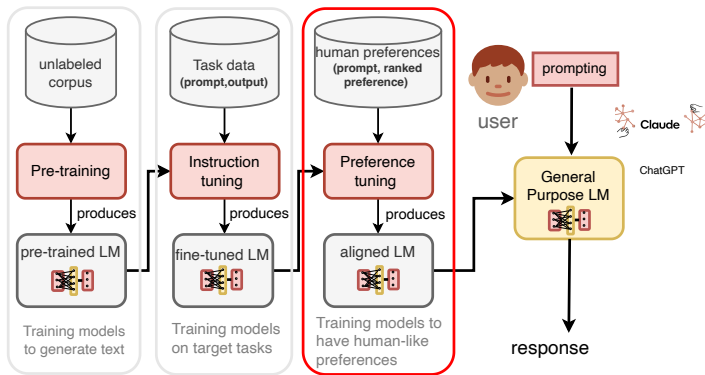
How do we get to general-purpose LLMs? **recipe**



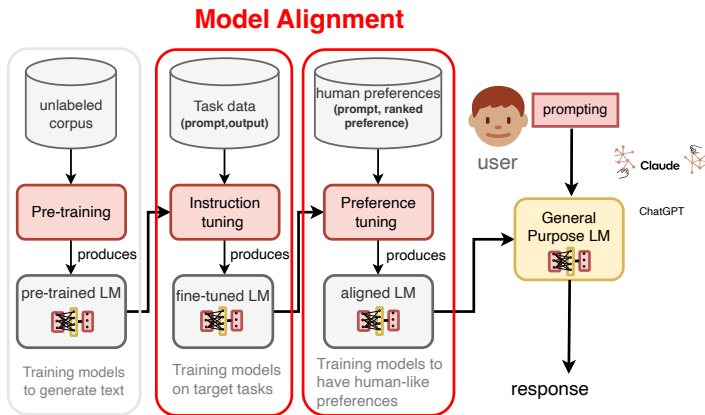
How do we get to general-purpose LLMs? **recipe**



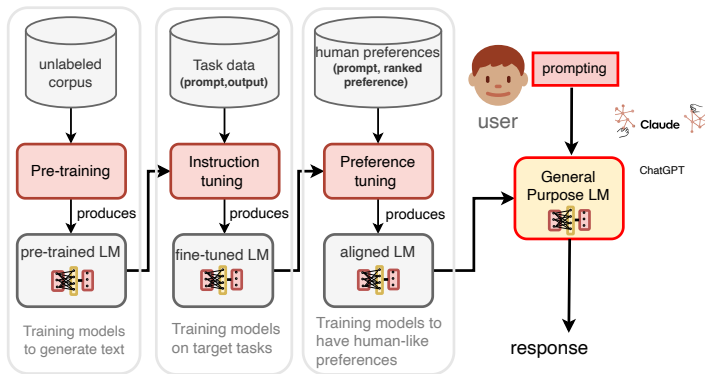
How do we get to general-purpose LLMs? **recipe**



How do we get to general-purpose LLMs? **recipe**



How do we get to general-purpose LLMs? **recipe**



Open model development: Olmo

Model Details



Model Card for Olmo 3 Think

We introduce Olmo 3, a new family of 7B and 32B models both Instruct and Think variants. Long chain-of-thought thinking improves reasoning tasks like math and coding.

Olmo is a series of Open language models designed to enable the science of language models. These models are pre-trained on the Dolma 3 dataset and post-trained on the Dolci datasets. We are releasing all code, checkpoints, logs (coming soon), and associated training details.

The core models released in this batch include the following:

Stage	Olmo 3 7B Think	Olmo 3 32B Think	Olmo 3 7B Instruct
Base Model	Olmo-3-7B	Olmo-3-32B	Olmo-3-7B
SFT	Olmo-3-7B-Think-SFT	Olmo-3-32B-Think-SFT	Olmo-3-7B-Instruct-SFT
DPO	Olmo-3-7B-Think-DPO	Olmo-3-32B-Think-DPO	Olmo-3-7B-Instruct-DPO
Final Models (RLVR)	Olmo-3-7B-Think	Olmo-3-32B-Think	Olmo-3-7B-Instruct

Downloads last month
61,316



Safetensors

Model size: **7B params** | Tensor type: **BF16** | Chat template | Files info

Inference Providers

Text Generation

This model isn't deployed by any inference Provider. [Ask for provider support](#)

Model tree for allenai/Olmo-3-7B-Think

Base model: [allenai/Olmo-3-1025-7B](#)

- Finetuned
 - Finetuned
 - Finetuned (7)
 - Adapters: 9 models
 - Finetunes: 11 models
 - Merges: 1 model
 - Quantizations: 32 models

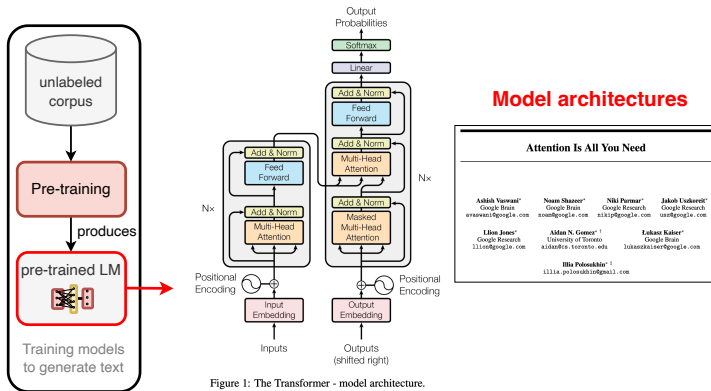
Dataset used to train allenai/Olmo-3-7B-Think

- [allenai/Dolci-Think-RL-7B](#)

[Viewer](#) • Updated Jan 5 • 102k • 89% • 17

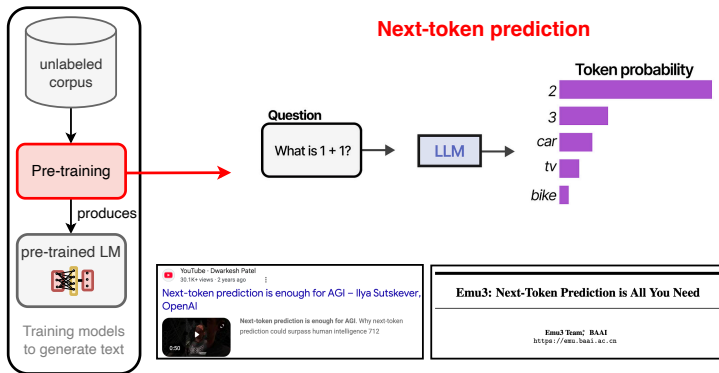
<https://huggingface.co/collections/allenai/olmo-3>

The landscape of generative AI research: **Foundations**



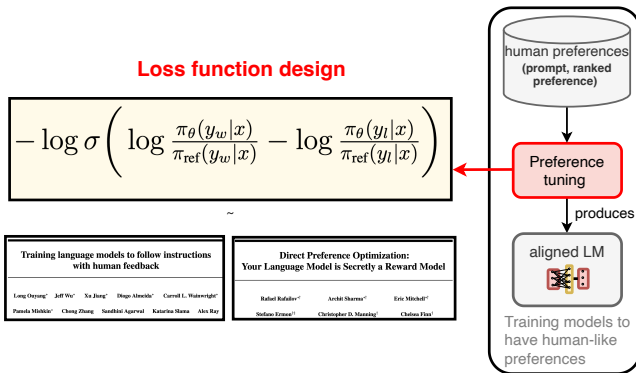
- **Model architecture design:** what kinds of problems can specific architectures (e.g., transformers) express? Are there clear limitations?

The landscape of generative AI research: **Foundations**



- **Pre-training and evaluation:** Why is next-token prediction such an effective signal and how do we effectively measure model knowledge?

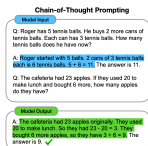
The landscape of generative AI research: **Alignment**



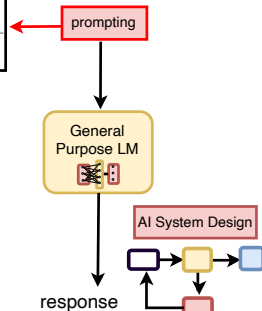
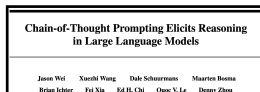
- **Loss design:** What do we do when we train models to preferences? Does the optimization have an internal logic?

What do these computations mean?

The landscape of generative AI research: Inference



Test-time inference



- **Test-time inference:** What kinds of reasoning structures do LLMs build? How can we know if the reasoning is correct and sound?

The landscape of generative AI research: Inference

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

Model Output

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Test-time inference

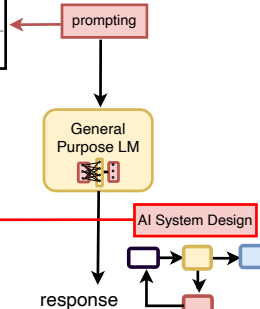
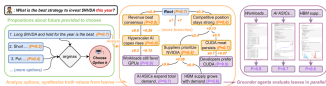
Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei	Xuechi Wang	Dale Schuurmans	Maarten Bosma
Brian Ichter	Fei Xia	Ed H. Chi	Qiao V. Le
		Denny Zhou	

LLM Agents

ANALYTICA: SOFT PROPOSITIONAL REASONING FOR ROBUST AND SCALABLE LLM-DRIVEN ANALYSIS

Junyan Cheng^{*} Kyle Richardson^{*} Peter Chin^{*}
 Dartmouth College^{*} Allen Institute for AI^{*}
jc.th@dartmouth.edu kyle.r@allenai.org
[GitHub Repo](#) [Live Demo](#)



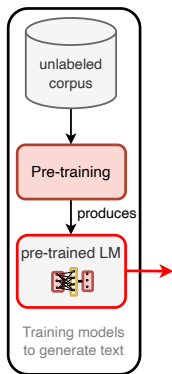
- ▶ **Test-time inference:** What kinds of reasoning structures do LLMs build? How can we know if the reasoning is correct and sound?

LLMs build structures, what are these structures?

The landscape of generative AI research: Inference

How are all these areas connected to one another?

Programming languages for LLM research



Restricted Access Sequence Processing Language (**Rasp**)

Thinking Like Transformers

Gail Weiss¹ Yoav Goldberg^{2,3} Eran Yahav¹

```

1 def num_prevs(bools) {
2   prevs = select(indices, indices, <=);
3   return (indices+1) *
4     aggregate(prevs,
5       indicator(bools));
6 }
7 n_opens = num_prevs(tokens=="(");
8 n_closes = num_prevs(tokens==")");
9 balance = n_opens - n_closes;
10 prev_imbalances = num_prevs(balance<0);
11 dyck1PTF = "F" if prev_imbalances > 0
12           else
13           ("T" if balance==0 else "P");

```

Figure 15: RASP code for computing Dyck-1-PTF with the parentheses (and).

Masked Hard-Attention Transformers Recognize Exactly the Star-Free Languages

Andy Yang
University of Notre Dame

David Chiang
University of Notre Dame

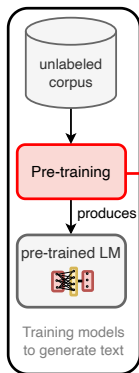
Dana Angluin
Yale University

$$\begin{aligned}
 P_\ell(i) &:= \blacktriangleright_j [j < i, 1] \ Q_\ell(j) : 0 \\
 S_r(i) &:= \blacktriangleleft_i [i > 1] \ O_r(i) : 0. \\
 I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
 B_\ell(i) &:= \blacktriangleright_j [j < i, \neg I(j)] \ Q_\ell(j) : 0 \\
 A_r(i) &:= \blacktriangleleft_j [j > i, \neg I(j)] \ Q_r(j) : 0 \\
 C(i) &:= I(i) \vee (O_r(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
 Y(i) &:= \blacktriangleright_j [1, \neg C(j)] \ 0 : 1.
 \end{aligned}$$

Figure 16: B-rasp for computing Dyck-1.

- **Rasp**: Languages that capture the underlying computational model of transformers (Weiss et al.2021; Yang and Chiang2024; Yang et al.2024; Huang et al.2026).

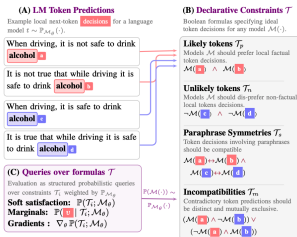
Programming languages for LLM research



ModelLog

From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models

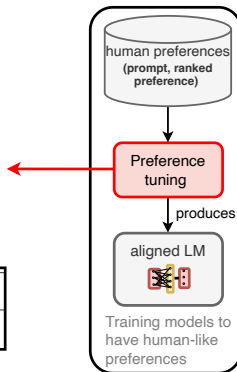
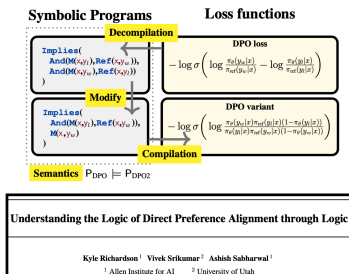
Kyle Richardson¹ Cullen Anderson² Pranav Balakrishnan² Takuto Ban² Daksha Ladia²
Ankita Gupta² Marisa Hudspeth²



- Declarative-style probabilistic programs, probabilistic logic, takes much inspiration from PLP (De Raedt and Kimmig2015; Fierens et al.2015; Manhaeve et al.2018).

Programming languages for LLM research

Declarative Loss Function Design



- ▶ Declarative-style probabilistic programs, probabilistic logic, takes much inspiration from PLP (De Raedt and Kimmig2015; Fierens et al.2015; Manhaeve et al.2018).

Languages provide a rich, declarative semantics

Programming languages for LLM research

LMQL

Prompting Is Programming: A Query Language for Large Language Models

LUCA BEURER-KELLNER, MARC FISCHER, and MARTIN VECHEV, ETH Zurich

```
qlml.query
def meaning_of_life():
    """
    # top-level strings are prompts
    "Q: What is the answer to life, the \
    universe and everything?"

    # generation via (constrained) variables
    "A: {ANSWER}!" where \
    len(ANSWER) < 120 and STOPS_AT(ANSWER, "!")

    # results are directly accessible
    print("LLM returned", ANSWER)

    # use typed variables for guaranteed
    # output format
    "The answer is {NUM: int}"

    # query programs are just functions
    return NUM

    # so from Python, you can just do this
    meaning_of_life() # 42
```

Copper

CuT as Trustable Probabilistic Programs

Alex Bahdanov University of Pennsylvania
David Rosenberg University of Pennsylvania
David Rosenberg University of Pennsylvania
David Rosenberg University of Pennsylvania

```
1 @CotPropParser()
2 def pp1():
3     P1: bool == Flip("step1", 0.86, [0.7])
4
5     P2: bool == Flip("step2", 0.96, [0.9])
6     ## observed(P) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool == Flip("no", 0.53, [0.8])
10        yield P3 ## return 'no' if P3 true, exception otherwise, syntactic sugar
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14 ## Program inference
15 pp.inference(query="no", compute_grad=True) ## 0.4376
16 ## param_grads(P2: 0.455, (P3: 0.825, (P1: 0.5087)
17 pp.inference(query=ValueError("no"), use_conf=True) ## 0.5624
18 pp.inference(query="no", use_conf=True) ## 0.504
```

prompting

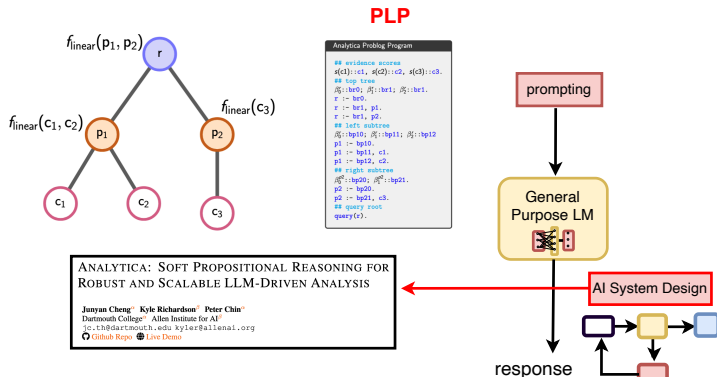
General Purpose LM

AI System Design

response

- ▶ Imperative-style programming, add structure to LLM workflows, mix with conventional programming (Beurer-Kellner et al.2023; Mandel et al.2026; Wu et al.2026).

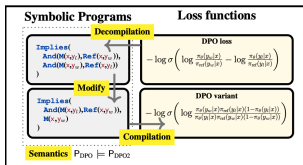
Programming languages for LLM research



- Modeling message passing, score aggregation in forecasting agents, LLM soft scoring; problog (De Raedt et al.2007; Dries et al.2015).

Tractable Language Model Programming

Language Model Programming



```
1 @CotProgParser()
2 def pp(i):
3     P1: bool == Flip("step1", 0.86, [0.7])
4
5     P2: bool == Flip("step2", 0.96, [0.9])
6     ## observed(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool == Flip("no", 0.33, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise, syntactic sugar
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14 ## Program inference
15 pp.inference(query="no", compute_grad=True) ## 0.4376
16 ## param_grad=(P2: 0.435, (P3: 0.825, (P1: 0.5087))
17 pp.inference(query="ValueError") ## 0.5024
18 pp.inference(query="no", use_conf=True) ## 0.504
```

```
1 @tool.query
2 def meaning_of_life():
3     """
4     # top-level strings are prompts
5     "Q: What is the answer to life, the \
6     universe and everything?"
7
8     # generation via (constrained) variables
9     "A: [ANSWER] where \
10     len(ANSWER) < 120 and STOPS_AT[ANSWER, "?"]
11
12     # results are directly accessible
13     print("LLM returned", ANSWER)
14
15     # use typed variables for guaranteed
16     # unique format
17     "The answer is {NUM: int}"
18
19     # query programs are just functions
20     return NUM
21
22 # so from Python, you can just do this
23 meaning_of_life() # 42
```

- **LM programming:** The study and design of languages for specifying, composing and reasoning about LLM computations.

Tractable Language Model Programming

Tractable Language Model Programming

Loss function design (Richardson et al.2025)

Understanding the Logic of Direct Preference Alignment through Logic

Kyle Richardson¹ Vivek Srikumar² Ashish Sabharwal¹

¹ Allen Institute for AI ² University of Utah
(kyler.ashish)@allenai.org vsivek@cs.utah.edu

Prompting (Richardson et al.2026b)

CoTs as Tractable Probabilistic Programs

Kyle Richardson [*] Allen Institute for Artificial Intelligence	Yu Feng [*] University of Pennsylvania	Poorva Garg University of California Los Angeles
Junyan Cheng Dartmouth College	Guy Van den Broeck University of California Los Angeles	Dan Roth University of Pennsylvania

Test-time inference (Garg et al.2026)

Probabilistic Programs of Thought

Poorva Garg
University of California Los Angeles
poorvagarg@cs.ucla.edu

Renato Lui Geh
University of California Los Angeles
renatolg@cs.ucla.edu

Daniel Israel
University of California Los Angeles
daniel.m.israel@cs.ucla.edu

Todd Millstein
University of California Los Angeles
todd@cs.ucla.edu

Kyle Richardson
Allen Institute for AI
kyler@allenai.org

Guy Van den Broeck
University of California Los Angeles
guyvdb@cs.ucla.edu

Pre-training evaluation (Richardson et al.2026a)

From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models

Kyle Richardson¹ Cullen Anderson² Pranav Balakrishnan² Takuto Ban² Daksha Ladia²
Ankita Gupta² Marisa Hudspeth²

Agentic modeling (Cheng et al.2026)

ANALYTICA: SOFT PROPOSITIONAL REASONING FOR ROBUST AND SCALABLE LLM-DRIVEN ANALYSIS

Junyan Cheng^{*} Kyle Richardson^{*} Peter Chin^{*}
Dartmouth College^{*} Allen Institute for AI^{*}

- ▶ **Tractable LM Programming:** languages designed for exact, tractable probabilistic and logical inference.

Tractable Language Model Programming

Loss function design (Richardson et al.2025)

Understanding the Logic of Direct Preference Alignment through Logic

Kyle Richardson¹ Vivek Srikumar² Ashish Sabharwal¹

¹ Allen Institute for AI ² University of Utah
(kyle.rashish)@allenai.org vsivek@cs.utah.edu

Prompting (Richardson et al.2026b)

CoTs as Tractable Probabilistic Programs

Kyle Richardson [*] Allen Institute for Artificial Intelligence	Yu Feng [*] University of Pennsylvania	Poorva Garg University of California Los Angeles
Junyan Cheng Dartmouth College	Guy Van den Broeck University of California Los Angeles	Dan Roth University of Pennsylvania

Test-time inference (Garg et al.2026)

Probabilistic Programs of Thought

Poorva Garg
University of California Los Angeles
poorvagarg@cs.ucla.edu

Renato Lui Geh
University of California Los Angeles
renatolg@cs.ucla.edu

Daniel Israel
University of California Los Angeles
daniel.m.israel@cs.ucla.edu

Todd Millstein
University of California Los Angeles
todd@cs.ucla.edu

Kyle Richardson
Allen Institute for AI
kyler@allenai.org

Guy Van den Broeck
University of California Los Angeles
guyvdb@cs.ucla.edu

Pre-training evaluation (Richardson et al.2026a)

From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models

Kyle Richardson¹ Cullen Anderson² Pranav Balakrishnan³ Takuto Ban² Daksha Ladia³
Ankita Gupta² Marisa Hudspeth²

Agentic modeling (Cheng et al.2026)

ANALYTICA: SOFT PROPOSITIONAL REASONING FOR ROBUST AND SCALABLE LLM-DRIVEN ANALYSIS

Junyan Cheng^{*} Kyle Richardson^{*} Peter Chin^{*}
Dartmouth College^{*} Allen Institute for AI^{*}

Today: discuss prompting and a discrete probabilistic programming language (COPPER) we built for modeling chain-of-thought.

CoTs as Tractable Probabilistic Programs

(with Yu Feng, Poorva Garg, Junyan Cheng, Guy Van den Broeck, Dan Roth)

What is chain-of-thought?

- ▶ Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

What is chain-of-thought?

- ▶ Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

What is chain-of-thought?

- ▶ Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

What is chain-of-thought?

- Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

Only law enforcement officers perform lawful arrests of other people.

s2

What is chain-of-thought?

- Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

Only law enforcement officers perform lawful arrests of other people.

s2

Based on this, the answer is no. (**answer: no**)

s3

What is chain-of-thought?

- Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

Only law enforcement officers perform lawful arrests of other people.

s2

Based on this, the answer is no. (**answer: no**)

s3

What is chain-of-thought?

- Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

Only law enforcement officers perform lawful arrests of other people.

s2

Based on this, the answer is no. (**answer: no**)

s3

where a **trace** $\mathbf{T} = (s_1, \dots, s_n)$ consisting of

What is chain-of-thought?

- ▶ Originally a prompting strategy that first asks models to **think step-by-step**, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

Only law enforcement officers perform lawful arrests of other people.

s2

Based on this, the answer is no. (**answer: no**)

s3

where a **trace** $\mathbf{T} = (s_1, \dots, s_n)$ consisting of

- ▶ A sequence of **thoughts** $s_i \sim \mathbb{P}_{\mathcal{M}}(\cdot \mid s_{<i}, Q)$.
- ▶ A **final answer** $A \sim \mathbb{P}_{\mathcal{M}}(\cdot \mid \mathbf{T}, Q)$.

What is chain-of-thought?

- Originally a prompting strategy that first asks models to think step-by-step, produce a reasoning trace before answering.

E.g.,

Prompt (Q) Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

Producing CoT is just decoding: $A, \mathbf{T} \sim \mathbb{P}_{\mathcal{M}}(\cdot \mid Q)$

Members of The Police are not law enforcement officers. 

Only law enforcement officers perform lawful arrests of other people. 

Based on this, the answer is no. (answer: no) 

Chain-of-thought is everywhere! It is a:

Prompting strategy

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Chain-of-thought is everywhere! It is a:

Prompting strategy

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

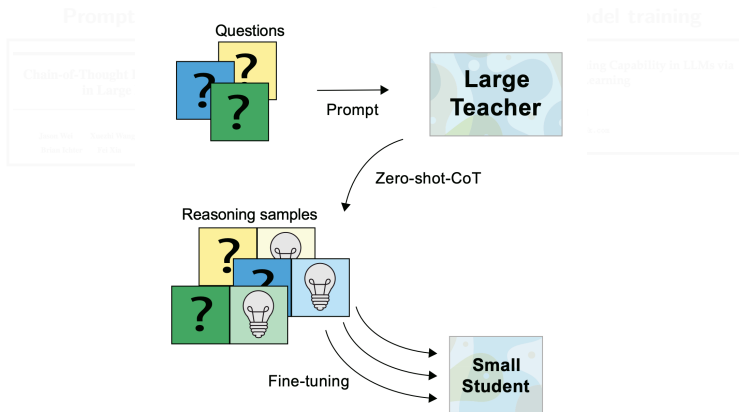
Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Structure for model training

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI
research@deepseek.com

Chain-of-thought is everywhere! It is a:



from Ho et al. (2023) *Large Language Models are Reasoning Teachers* ACL

Chain-of-thought is everywhere! It is a:

Prompting strategy

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Structure for model training

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI
research@deepseek.com

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao Dian Yu Jeffrey Zhao Izhak Shafran
Princeton University Google DeepMind Google DeepMind Google DeepMind

Thomas L. Griffiths Yuan Cao Karthik Narasimhan
Princeton University Google DeepMind Princeton University

Structure for test-time inference

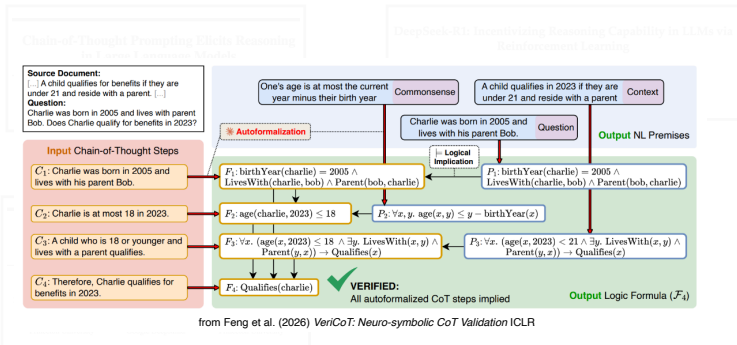


Key novelty (beyond CoT-SC): Generating several new thoughts based on a given arbitrary thought, exploring it further, and possibly backtracking from it

Chain-of-thought is everywhere! It is a:

Prompting strategy

Structure for model training



Structure for test-time inference

Object for behavioral analysis

Chain-of-thought is everywhere! It is a:

Prompting strategy

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Structure for model training

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI
research@deepseek.com

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao Dian Yu Jeffrey Zhao Izhak Shafran
Princeton University Google DeepMind Google DeepMind Google DeepMind
Thomas L. Griffiths Yuan Cao Karthik Narasimhan
Princeton University Google DeepMind Princeton University

Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting

Miles Turpin,^{1,2} Julian Michael,¹ Ethan Perez,^{1,3} Samuel R. Bowman^{1,3}
¹NYU Alignment Research Group, ²Cohere, ³Anthropic

Structure for test-time inference

Object for behavioral analysis

Chain-of-thought is everywhere! It is a:

Prompting strategy

Structure for model training



	In-context Demonstration	Inference by LLM
Query	Leah had 32 chocolates and her sister had 42. If they ate 35, how many pieces do they have left in total?	Julie is reading a 120-page book. Yesterday, she was able to read 12 pages and today, she read twice as many pages as yesterday. If she wants to read half of the remaining pages tomorrow, how many pages should she read?
Standard	The answer is 39.	The answer is 18. ✗
CoT	Originally, Leah had 32 chocolates and her sister had 42. So in total they had $32 + 42 = 74$. After eating 35, they had $74 - 35 = 39$ pieces left in total. The answer is 39.	Julie is reading a 120-page book. Yesterday, she read 12 pages and today, she read 24 pages. So she read a total of $12 + 24 = 36$ pages. Now she has $120 - 36 = 84$ pages left. Since she wants to read half of the remaining pages, she should read $84 / 2 = 42$ pages. The answer is 42. ✓
Invalid Reasoning	Originally, Leah had 32 chocolates and her sister had 42. So her sister had $42 - 32 = 10$ chocolates more than Leah has. After eating 35, since $10 + 35 = 45$, they had $45 - 6 = 39$ pieces left in total. The answer is 39.	Yesterday, Julie read 12 pages. Today, she read $12 * 2 = 24$ pages. So she read a total of $12 + 24 = 36$ pages. Now she needs to read $120 - 36 = 84$ more pages. She wants to read half of the remaining pages tomorrow, so she needs to read $84 / 2 = 42$ pages tomorrow. The answer is 42. ✓

g Capability in LLMs via



g What They Think: Chain-of-Thought

g Support R. Research? Chain-of-Thought

Structure for

from Wang et al. (2023) Towards Understanding CoT Prompting ACL

ral analysis

All of these areas are disconnected

Prompting strategy

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Structure for model training

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI
research@deepseek.com

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao Dian Yu Jeffrey Zhao Izhak Shafran
Princeton University Google DeepMind Google DeepMind Google DeepMind
Thomas L. Griffiths Yuan Cao Karthik Narasimhan
Princeton University Google DeepMind Princeton University

Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting

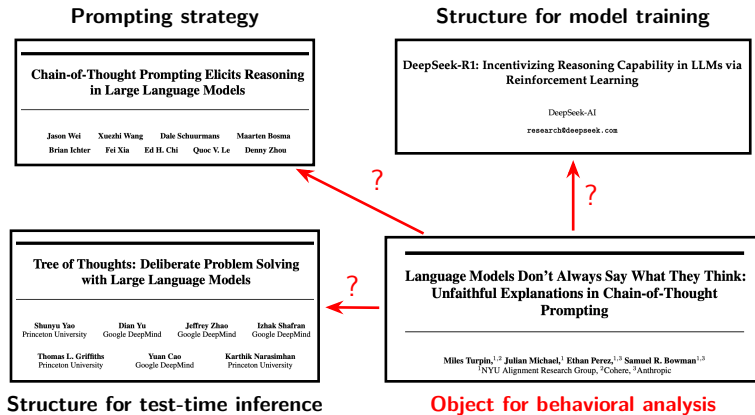
Miles Turpin,^{1,2} Julian Michael,¹ Ethan Perez,^{1,3} Samuel R. Bowman^{1,3}
¹NYU Alignment Research Group, ²Cohere, ³Anthropic

Structure for test-time inference

Object for behavioral analysis

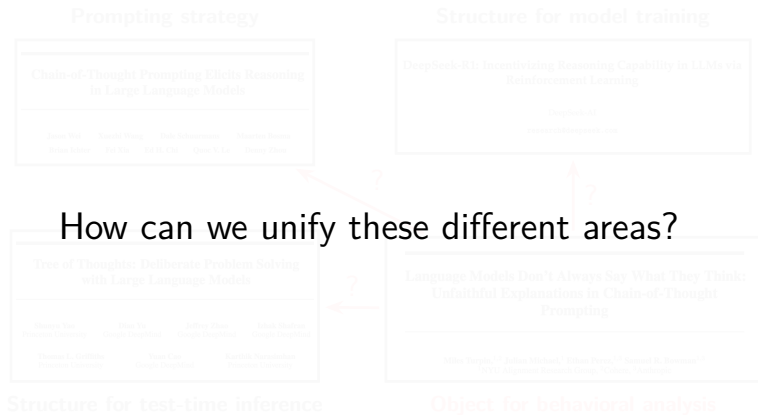
- **Problem:** All make different assumptions about what CoT is.

All of these areas are disconnected



► **Problem:** All make different assumptions about what CoT is.

All of these areas are disconnected



► **Problem:** All make different assumptions about what CoT is.

Making CoT a program

Making CoT a program

(A) Chain-of-thought

Prompt: Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians.

s1

deps. none likelihood: 0.86 confidence: 0.7

Only law enforcement officers perform lawful arrests of other people.

s2

deps. none likelihood: 0.96 confidence: 0.90

Based on this, the answer is no. (answer: no)

s3

deps. s1, s2 likelihood: 0.53 confidence: 0.8

Making CoT a program

(A) Chain-of-thought

Prompt: Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians. s1

deps. none likelihood: 0.86 confidence: 0.7

Only law enforcement officers perform lawful arrests of other people. s2

deps. none likelihood: 0.96 confidence: 0.90

Based on this, the answer is no. **(answer: no)** s3

deps. s1, s2 likelihood: 0.53 confidence: 0.8

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("step1", 0.86, [0.7])
4
5     P2: bool =~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool =~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

Making CoT a program

(A) Chain-of-thought

Prompt: Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians. s1

deps. none likelihood: 0.86 confidence: 0.7

Only law enforcement officers perform lawful arrests of other people. s2

deps. none likelihood: 0.96 confidence: 0.90

Based on this, the answer is no. (answer: no) s3

deps. s1, s2 likelihood: 0.53 confidence: 0.8

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool = ~ Flip("step1", 0.86, [0.7])
4
5     P2: bool = ~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool = ~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

Assumptions about $\mathbf{T} = (s_1, \dots, s_n)$:

1. Steps s_j are discrete objects, propositions; probabilistic (θ).

Making CoT a program

(A) Chain-of-thought

Prompt: Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians. s1

deps. none likelihood: 0.86 confidence: 0.7

Only law enforcement officers perform lawful arrests of other people. s2

deps. none likelihood: 0.96 confidence: 0.90

Based on this, the answer is no. (answer: no) s3

deps. s1, s2 likelihood: 0.53 confidence: 0.8

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool = ~ Flip("step1", 0.86, [0.7])
4
5     P2: bool = ~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool = ~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

Assumptions about $\mathbf{T} = (s_1, \dots, s_n)$:

1. Steps s_j are discrete objects, propositions; probabilistic (θ).
2. Step dependencies are if-then control flow

Making CoT a program

(A) Chain-of-thought

Prompt: Could the members of The Police perform lawful arrests? Think step-by-step.

LLM CoT Reasoning Trace:

The members of The Police were musicians. s1
deps. none likelihood: 0.86 confidence: 0.7

Only law enforcement officers perform lawful arrests of other people. s2
deps. none likelihood: 0.96 confidence: 0.90

Based on this, the answer is no. (answer: no) s3
deps. s1, s2 likelihood: 0.53 confidence: 0.8

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool = ~ Flip("step1", 0.86, [0.7])
4
5     P2: bool = ~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool = ~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

Assumptions about $\mathbf{T} = (s_1, \dots, s_n)$:

1. Steps s_j are discrete objects, propositions; probabilistic (θ).
2. Step dependencies are if-then control flow
3. Answers are return values, reasoning failures are exceptions.

CoT as a probabilistic program

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("step1", 0.86, [0.7])
4
5     P2: bool =~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool =~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

(C) Probabilistic Queries

$$\begin{aligned}\theta_{\text{like.}} &= \\ & (0.86, 0.96, 0.53) \\ \theta' &= (0.70, 0.90, 0.80)\end{aligned}$$

- Questions about a CoT trace **T** become familiar probabilistic queries, e.g., marginals, sensitivities, conditionals, robustness as weight perturbations.

CoT as a probabilistic program

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("step1", 0.86, [0.7])
4
5     P2: bool =~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool =~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

(C) Probabilistic Queries

$$\begin{aligned}\theta_{\text{like.}} &= \\ (0.86, 0.96, 0.53) \\ \theta' &= (0.70, 0.90, 0.80)\end{aligned}$$

$\mathbb{P}_{pp}(\text{return "no"; } \theta_{\text{like.}})$

$\mathbb{P}_{pp}(\text{raise ValueError; } \theta_{\text{like.}})$

- Questions about a CoT trace **T** become familiar probabilistic queries, e.g., marginals, sensitivities, conditionals, robustness as weight perturbations.

CoT as a probabilistic program

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("step1", 0.86, [0.7])
4
5     P2: bool =~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool =~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

(C) Probabilistic Queries

$$\begin{aligned}\theta_{\text{like.}} &= \\ (0.86, 0.96, 0.53) \\ \theta' &= (0.70, 0.90, 0.80)\end{aligned}$$

$$\mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like.}})$$

$$\mathbb{P}_{\text{pp}}(\text{raise ValueError; } \theta_{\text{like.}})$$

$$\frac{\partial \mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like.}})}{\partial \theta_{\text{like.}}}$$

- Questions about a CoT trace **T** become familiar probabilistic queries, e.g., marginals, sensitivities, conditionals, robustness as weight perturbations.

CoT as a probabilistic program

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool = ~ Flip("step1", 0.86, [0.7])
4
5     P2: bool = ~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool = ~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

(C) Probabilistic Queries

$$\begin{aligned}\theta_{\text{like.}} &= \\ (0.86, 0.96, 0.53) \\ \theta' &= (0.70, 0.90, 0.80)\end{aligned}$$

$$\mathbf{E} = \{\text{P2}\}$$

$$\mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like.}})$$

$$\mathbb{P}_{\text{pp}}(\text{raise ValueError; } \theta_{\text{like.}})$$

$$\frac{\partial \mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like.}})}{\partial \theta_{\text{like.}}}$$

- Questions about a CoT trace **T** become familiar probabilistic queries, e.g., marginals, sensitivities, conditionals, robustness as weight perturbations.

CoT as a probabilistic program

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("step1", 0.86, [0.7])
4
5     P2: bool =~ Flip("step2", 0.96, [0.9])
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         P3: bool =~ Flip("no", 0.53, [0.8])
10        yield P3 ## returns 'no' if P3 true, exception otherwise
11    else:
12        raise ValueError("Cannot establish P1 or P2")
13
14    ## Program inference
15    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
16    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
17    pp.inference(query={"ValueError"}) ## 0.5624
18    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

(C) Probabilistic Queries

$$\begin{aligned}\theta_{\text{like.}} &= \\ (0.86, 0.96, 0.53) \\ \theta' &= (0.70, 0.90, 0.80)\end{aligned}$$

$$\mathbf{E} = \{\mathbf{P2}\}$$

$$\mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like.}})$$

$$\mathbb{P}_{\text{pp}}(\text{raise ValueError; } \theta_{\text{like.}})$$

$$\frac{\partial \mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like.}})}{\partial \theta_{\text{like.}}}$$

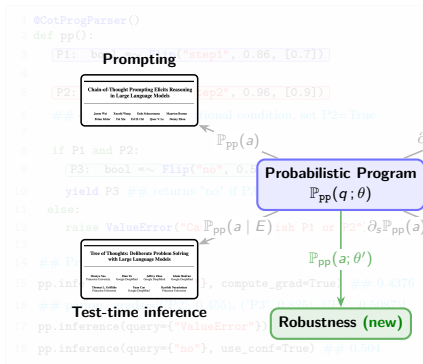
$$\mathbb{P}_{\text{pp}}(\text{return "no"; } \theta')$$

(new)

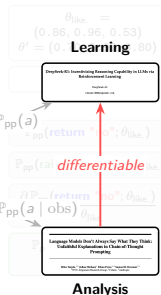
- Questions about a CoT trace **T** become familiar probabilistic queries, e.g., marginals, sensitivities, conditionals, robustness as weight perturbations.

CoT as a probabilistic program

(B) Probabilistic Program



(C) Probabilistic Queries



CoT as a probabilistic program

(B) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     [P1: bool ~ Flip("step1", 0.86, [0.7])]
4
5     [P2: bool ~ Flip("step2", 0.96, [0.9])]
6     ## observe(P2) ## Optional condition, set P2=True
7
8     if P1 and P2:
9         [P3: bool ~ Flip("no", 0.53, [0.8])]
10    else:
11        raise ValueError("Cannot establish P1 or P2")
12
13    ## Program inference
14    pp.inference(query={"no"}, compute_grad=True) ## 0.4376
15    ## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
16    pp.inference(query={"ValueError"}) ## 0.5624
17    pp.inference(query={"no"}, use_conf=True) ## 0.504
```

(C) Probabilistic Queries

$$\begin{aligned}\theta_{\text{like}} &= \\ &(0.86, 0.96, 0.53) \\ \theta' &= (0.70, 0.90, 0.80)\end{aligned}$$

$$\mathcal{E} = \{P2\}$$

$$\mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like}})$$

$$\begin{aligned}\mathbb{P}_{\text{pp}}(\text{raise ValueError; } \theta_{\text{like}}) \\ \frac{\partial \mathbb{P}_{\text{pp}}(\text{return "no"; } \theta_{\text{like}})}{\partial \theta_{\text{like}}}\end{aligned}$$

$$\begin{aligned}\mathbb{P}_{\text{pp}}(\text{return "no"; } \theta') \\ (\text{new})\end{aligned}$$

What programming language do we need?

The Copper Language: Syntax

CoT Concept	Syntax
Steps are probabilistic Booleans	1 <code>P =~ Flip(<str>, <p>)</code>
Step dependencies are controls	1 <code>if <bool condition>:</code> 2 <new fact or answer branch v>
Reasoning Errors are Exceptions	1 <code>if <bool condition>:</code> 2 <new fact or answer branch v> 3 <code>else:</code> 4 <code>raise ValueError(<error>)</code>
Answers are returns	1 <code>return <answer output></code>
Conditions are observes	1 <code>observe(<bool condition e>)</code>
Chains/answers can be unique	1 <code>A₁, ..., A_n =~ Categorical(<p₁, ...>)</code>
Step scores can be aggregated	1 <code>P =~ Factor(<str>, f_m(p₁, ..., p_m))</code>

- **Discrete probabilistic programming**, limited to discrete distributions and Boolean variables, special features for step aggregation.

A simplified fragment of Dice

Scaling Exact Inference for Discrete Probabilistic Programs

STEVEN HOLTZEN, University of California, Los Angeles

GUY VAN DEN BROECK, University of California, Los Angeles

TODD MILLSTEIN, University of California, Los Angeles

Probabilistic programming languages (PPLs) are an expressive means of representing and reasoning about probabilistic models. The computational challenge of *probabilistic inference* remains the primary roadblock for applying PPLs in practice. Inference is fundamentally hard, so there is no one-size-fits all solution. In this work, we target scalable inference for an important class of probabilistic programs: those whose probability distributions are *discrete*. Discrete distributions are common in many fields, including text analysis, network verification, artificial intelligence, and graph analysis, but they prove to be challenging for existing PPLs.

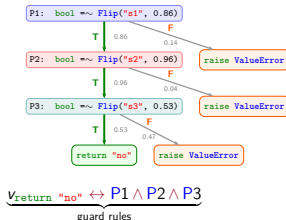
We develop a domain-specific probabilistic programming language called Dice that features a new approach to exact discrete probabilistic program inference. Dice exploits program structure in order to *factorize* inference, enabling us to perform exact inference on probabilistic programs with hundreds of thousands of random variables. Our key technical contribution is a new reduction from discrete probabilistic programs to *weighted model counting* (WMC). This reduction separates the structure of the distribution from its parameters, enabling logical reasoning tools to exploit that structure for probabilistic inference. We (1) show how to compositionally reduce Dice inference to WMC, (2) prove this compilation correct with respect to a denotational semantics, (3) empirically demonstrate the performance benefits over prior approaches, and (4) analyze the types of structure that allow Dice to scale to large probabilistic programs.

The Copper Language: Query Computation

(A) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("s1", 0.86)
4     P2: bool =~ Flip("s2", 0.96)
5     if P1 and P2:
6         P3: bool =~ Flip("s3", 0.53)
7         if P3: return "no"
8         else: raise ValueError
9     else:
10        raise ValueError
```

(B) Reachability semantics

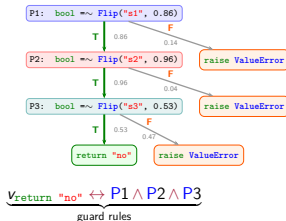


The Copper Language: Query Computation

(A) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3     P1: bool =~ Flip("s1", 0.86)
4     P2: bool =~ Flip("s2", 0.96)
5     if P1 and P2:
6         P3: bool =~ Flip("s3", 0.53)
7         if P3: return "no"
8         else: raise ValueError
9     else:
10        raise ValueError
```

(B) Reachability semantics



► Encode programs as Boolean formulas:

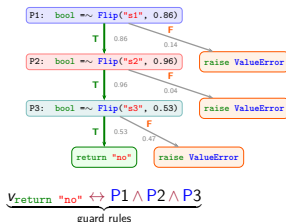
$$F := \bigwedge_{v \in \mathcal{V}_{\text{det}}} \left(v \leftrightarrow \bigvee_{g \in \text{guards}(v)} g \right) \quad E := \bigwedge_{e \in \text{evidence}} e$$

The Copper Language: Query Computation

(A) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3   P1: bool =~ Flip("s1", 0.86)
4   P2: bool =~ Flip("s2", 0.96)
5   if P1 and P2:
6     P3: bool =~ Flip("s3", 0.53)
7     if P3: return "no"
8     else: raise ValueError
9   else:
10    raise ValueError
```

(B) Reachability semantics



► Encode programs as Boolean formulas:

$$F := \bigwedge_{v \in \mathcal{V}_{\text{det}}} \left(v \leftrightarrow \bigvee_{g \in \text{guards}(v)} g \right) \quad E := \bigwedge_{e \in \text{evidence}} e$$

with query q probabilities computed as:

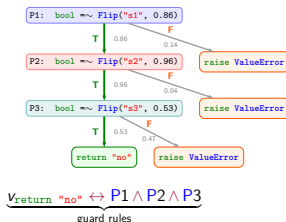
$$\mathbb{P}_F(q \mid E; \theta) = \frac{\text{WMC}(F \wedge E \wedge q; \theta)}{\text{WMC}(F \wedge E; \theta)}$$

The Copper Language: Query Computation

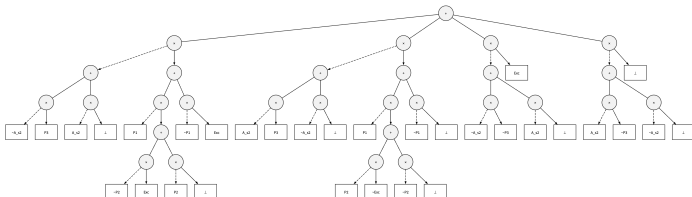
(A) Probabilistic Program

```
1 @CotProgParser()
2 def pp():
3   P1: bool =~ Flip("s1", 0.86)
4   P2: bool =~ Flip("s2", 0.96)
5   if P1 and P2:
6     P3: bool =~ Flip("s3", 0.53)
7     if P3: return "no"
8     else: raise ValueError
9   else:
10    raise ValueError
```

(B) Reachability semantics



► Exact WMC inference via circuits (pySDD):



```
1 @CotProgParser()
2 def pp():
3   P1: bool ~ P
4   P2: bool ~ P
5   if P1 and P2:
6     P3: bool ~ P
7     if P3: raise Valuck
8   else: raise
9 else:
10  raise Valuck
```

Inference and Learning in Probabilistic Logic Programs using Weighted Boolean Formulas

DAAN FIERENS, GUY VAN DEN BROECK, JORIS RENKENS,
DIMITAR SHTERIONOV, BERND GUTMANN, INGO THON,
GERDA JANSSENS, LUC DE RAEDT

Department of Computer Science
KU Leuven
Celestijnenlaan 300A, 3001 Heverlee, Belgium
(e-mail: FirstName.LastName@cs.kuleuven.be)

submitted 26 June 2012; revised 04 January 2013; accepted 07 March 2013

Abstract

Probabilistic logic programs are logic programs in which some of the facts are annotated with probabilities. This paper investigates how classical inference and learning tasks known from the graphical model community can be tackled for probabilistic logic programs. Several such tasks such as computing the marginals given evidence and learning from (partial) interpretations have not really been addressed for probabilistic logic programs before.

The first contribution of this paper is a suite of efficient algorithms for various inference tasks. It is based on a conversion of the program and the queries and evidence to a weighted Boolean formula. This allows us to reduce the inference tasks to well-studied tasks such as weighted model counting, which can be solved using state-of-the-art methods known from the graphical model and knowledge compilation literature. The second contribution is an algorithm for parameter estimation in the learning from interpretations setting. The algorithm employs Expectation Maximization, and is built on top of the developed inference algorithms.

The proposed approach is experimentally evaluated. The results show that the inference algorithms improve upon the state-of-the-art in probabilistic logic programming and that it is indeed possible to learn the parameters of a probabilistic logic program from interpretations.



The Copper Language: Query Computation

(A) Probabilistic Program

```
1 SCotProgParser()
2 def pp():
3     P1: bool ~ Flip("a1", 0.86)
4     P2: bool ~ Flip("a2", 0.96)
5     if P1 and P2:
6         P3: bool ~ Flip("a3", 0.53)
7         if P3: return "No"
8         else: raise ValueError
9     else: raise ValueError
10
```

(B) Reachability semantics



Program "No" $\leftrightarrow$ $P1 \wedge P2 \wedge P3$

What can we do with this simple language?

Distilling LLM computations to probabilistic programs

```
1 ## Answer 1
2  $P_1 \sim \text{Flip}(s_1, \theta_1)$ 
  ...
3  $P_n \sim \text{Flip}(s_n, \theta_n)$ 
4 if  $P_1$  and ... and  $P_n$ :
5     return a
6 else:
7     raise ValueError('Error')
```

Distilling LLM computations to probabilistic programs

CoT trace
 $T^a = (s_1, \dots, s_n)$

```
1 ## Answer 1
2 P1 = ~ Flip(s1, θ1)
   ...
3 Pn = ~ Flip(sn, θn)
4 if P1 and ... and Pn:
5     return a
6 else:
7     raise ValueError('Error')
```

Distilling LLM computations to probabilistic programs

CoT trace

$$\mathbf{T}^a = (s_1, \dots, s_n)$$

```
1 ## Answer 1
2 P1 = ~ Flip(s1, θ1)
  ...
3 Pn = ~ Flip(sn, θn)
4 if P1 and ... and Pn:
5     return a
6 else:
7     raise ValueError('Error')
```

LLM weighting

$$\theta_j = \mathbb{P}_{\mathcal{M}}(s_j | s_{<j}, Q)$$

Distilling LLM computations to probabilistic programs

CoT trace
 $\mathbf{T}^a = (s_1, \dots, s_n)$

```
1 ## Answer 1
2 P1 = ~ Flip(s1, θ1)
  ...
3 Pn = ~ Flip(sn, θn)
4 if P1 and ... and Pn:
5     return a
6 else:
7     raise ValueError('Error')
```

LLM weighting
 $\theta_j = \mathbb{P}_{\mathcal{M}}(s_j | s_{<j}, Q)$

- ▶ The probability of not raising an exception is equal to the LLM's CoT likelihood, i.e.,

$$\mathbb{P}_{\text{pp}}(\neg \text{raise ValueError}; \theta) = \mathbb{P}_{\mathcal{M}}(\mathbf{T}^a | Q)$$

Distilling LLM computations to probabilistic programs

CoT trace
 $\mathbf{T}^a = (s_1, \dots, s_n)$

```
1 ## Answer 1
2 P1 = ~ Flip(s1, θ1)
  ...
3 Pn = ~ Flip(sn, θn)
4 if P1 and ... and Pn:
5     return a
6 else:
7     raise ValueError('Error')
```

LLM weighting
 $\theta_j = \mathbb{P}_{\mathcal{M}}(s_j | s_{<j}, Q)$

Correspondingly,

$$-\log \mathbb{P}_{\text{pp}}(\neg \text{raise } \text{ValueError}; \theta)$$

is equal to the negative log-likelihood loss, giving semantics to the standard training loss.

Modeling multiple traces

$$\mathbf{T}^{a_1} = (s_{1,1}, \dots, s_{1,n_1})$$

$$\mathbf{T}^{a_k} = (s_{k,1}, \dots, s_{k,n_k})$$

```
1 P1,1 =~ Flip(s1,1, θ1,1) ...
```

```
2 P1,n1 =~ Flip(s1,n1, θ1,n1)
```

```
...
```

```
3 Pk,1 =~ Flip(sk,1, θk,1) ...
```

```
4 Pk,nk =~ Flip(sk,nk, θk,nk)
```

(A) Independent branches

```
5 if P1,1 and ... and P1,n1:  
6     return a1  
    ...  
7 elif Pk,1 and ... and Pk,nk:  
8     return ak  
9 else:  
10    raise ValueError
```

(B) Categorical branching

```
5 A1, ..., Ak, ¬A =~ Categorical(  
6     prod(θ1,: ), ..., prod(θk,: ),  
7     1 - (prod(θ1,: ) + ... + prod(θk,: ))  
8 )  
9 if A1:     return a1 ...  
10 elif Ak:  return ak  
11 else:     raise ValueError
```

Modeling multiple traces

$$\mathbf{T}^{a_1} = (s_{1,1}, \dots, s_{1,n_1})$$

$$\mathbf{T}^{a_k} = (s_{k,1}, \dots, s_{k,n_k})$$

```
1  $P_{1,1} \sim \text{Flip}(s_{1,1}, \theta_{1,1}) \dots$ 
```

```
2  $P_{1,n_1} \sim \text{Flip}(s_{1,n_1}, \theta_{1,n_1})$ 
```

```
...
```

```
3  $P_{k,1} \sim \text{Flip}(s_{k,1}, \theta_{k,1}) \dots$ 
```

```
4  $P_{k,n_k} \sim \text{Flip}(s_{k,n_k}, \theta_{k,n_k})$ 
```

(A) Independent branches

```
5 if  $P_{1,1}$  and ... and  $P_{1,n_1}$ :  
6     return  $a_1$   
    ...  
7 elif  $P_{k,1}$  and ... and  $P_{k,n_k}$ :  
8     return  $a_k$   
9 else:  
10    raise ValueError
```

(B) Categorical branching

```
5  $A_1, \dots, A_k, \neg A \sim \text{Categorical}(\$   
6      $\text{prod}(\theta_{1,:}), \dots, \text{prod}(\theta_{k,:}),$   
7      $1 - (\text{prod}(\theta_{1,:}) + \dots + \text{prod}(\theta_{k,:}))$   
8 )  
9 if  $A_1$ :     return  $a_1 \dots$   
10 elif  $A_k$ :  return  $a_k$   
11 else:      raise ValueError
```

► Here : $\mathbb{P}_{\text{pp}}(\neg \text{raise ValueError}; \theta)$ models

A: the probability that the LLM produces at least one k chain;

B: their total probability mass.

Modeling multiple traces

$$\mathbf{T}^{a_1} = (s_{1,1}, \dots, s_{1,n_1})$$

```
1 P1,1 =~ Flip(s1,1, θ1,1) ...  
2 P1,n1 =~ Flip(s1,n1, θ1,n1)  
...
```

$$\mathbf{T}^{a_k} = (s_{k,1}, \dots, s_{k,n_k})$$

```
3 Pk,1 =~ Flip(sk,1, θk,1) ...  
4 Pk,nk =~ Flip(sk,nk, θk,nk)
```

(A) Independent branches

```
5 if P1,1 and ... and P1,n1:  
6     return a1  
    ...  
7 elif Pk,1 and ... and Pk,nk:  
8     return ak  
9 else:  
10    raise ValueError
```

(B) Categorical branching

```
5 A1, ..., Ak, ¬A =~ Categorical(  
6     prod(θ1,:}), ..., prod(θk,:}),  
7     1 - (prod(θ1,:}) + ... + prod(θk,:}))  
8 )  
9 if A1:     return a1 ...  
10 elif Ak:  return ak  
11 else:     raise ValueError
```

Special case: when $a_1 = \dots = a_k$, **(B)** finitely approximates the standard CoT generative model (Dohan et al.2022), self-consistency (Wang et al.2022):

$$P_{\text{CoT}}(A \mid Q) = \sum_{\mathbf{T}} \mathbb{P}_{\mathcal{M}}(A, \mathbf{T} \mid Q)$$

Modeling multiple traces

$T^{a_1} = (s_{1,1}, \dots, s_{1,m_1})$

$T^{a_k} = (s_{k,1}, \dots, s_{k,n_k})$

```
1  $P_{1,1} \sim \text{Flip}(s_{1,1}, \theta_{1,1}) \dots$ 
```

```
2  $P_{1,m_1} \sim \text{Flip}(s_{1,m_1}, \theta_{1,m_1})$ 
```

```
...
```

```
3  $P_{k,1} \sim \text{Flip}(s_{k,1}, \theta_{k,1}) \dots$ 
```

```
4  $P_{k,n_k} \sim \text{Flip}(s_{k,n_k}, \theta_{k,n_k})$ 
```

(A) Independent branches

(B) Categorical branching

```
5 if  $P_{1,1}$  and ... and  $P_{1,m_1}$ :
```

```
...
```

```
7 elif  $P_{k,1}$  and ... and  $P_{k,n_k}$ :
```

```
8     return  $a_k$ 
```

```
9 else:
```

```
10     raise ValueError
```

```
5  $A_1, \dots, A_k, \neg A \sim \text{Categorical}(\theta_1, \dots, \theta_k, 1 - (\text{prod}(\theta_1, \dots) + \dots + \text{prod}(\theta_k, \dots)))$ 
```

```
7  $1 - (\text{prod}(\theta_1, \dots) + \dots + \text{prod}(\theta_k, \dots))$ 
```

```
8 )
```

```
9 if  $A_1$ :     return  $a_1 \dots$ 
```

```
10 elif  $A_k$ :  return  $a_k$ 
```

```
11 else:     raise ValueError
```

Standard CoT computations fall out as special cases

Modeling multiple traces

$T^{a_1} = (s_{1,1}, \dots, s_{1,m_1})$

```
1 P1,1 ~ Flip(s1,1, θ1,1) ...
```

```
2 P1,m1 ~ Flip(s1,m1, θ1,m1)
```

...

$T^{a_k} = (s_{k,1}, \dots, s_{k,n_k})$

```
3 Pk,1 ~ Flip(sk,1, θk,1) ...
```

```
4 Pk,nk ~ Flip(sk,nk, θk,nk)
```

(A) Independent branches

(B) Categorical branching

```
5 if P1,1 and ... and P1,m1:
```

```
6 ...
```

```
7 elif Pk,1 and ... and Pk,nk:
```

```
8     return ak
```

```
9 else:
```

```
10     raise ValueError
```

```
5 A1, ..., Ak, -A ~ Categorical(
```

```
7     1 - (prod(θ1,1) + ... + prod(θk,nk))
```

```
8 )
```

```
9 if A1:     return a1 ...
```

```
10 elif Ak:  return ak
```

```
11 else:     raise ValueError
```

note: We know how to do these computations

Going from programs to LLM computations

Likelihood $\mathbb{P}_\wedge$

1 $P_1 \sim \text{Flip}(s_1, \theta_1) \cdots$

2 $P_n \sim \text{Flip}(s_n, \theta_n)$

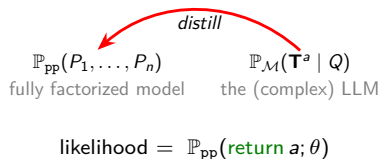
3 **if** P_1 **and** $\cdots$ **and** P_n :

4 **return** a

Going from programs to LLM computations

Likelihood $\mathbb{P}_\wedge$

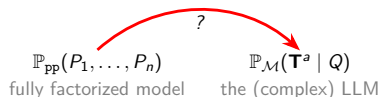
```
1  $P_1 \sim \text{Flip}(s_1, \theta_1) \dots$   
2  $P_n \sim \text{Flip}(s_n, \theta_n)$   
  
3 if  $P_1$  and  $\dots$  and  $P_n$ :  
4   return a
```



Going from programs to LLM computations

Likelihood $\mathbb{P}_\wedge$

```
1  $P_1 = \sim \text{Flip}(s_1, \theta_1) \dots$   
2  $P_n = \sim \text{Flip}(s_n, \theta_n)$   
  
3 if  $P_1$  and  $\dots$  and  $P_n$ :  
4   return a
```



likelihood = $\mathbb{P}_{\text{pp}}(\text{return } a; \theta)$

Going from programs to LLM computations

Likelihood $\mathbb{P}_{\wedge}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 and ... and Pn:  
4   return a
```

Modified $\mathbb{P}_{\vee}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 or ... or Pn:  
4   return a
```

Going from programs to LLM computations

Likelihood $\mathbb{P}_\wedge$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 and ... and Pn:  
4   return a  
  
return a ⇔ P1 ∧ ... ∧ Pn
```

Modified $\mathbb{P}_\vee$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 or ... or Pn:  
4   return a  
  
return a ⇔ P1 ∨ ... ∨ Pn
```

Going from programs to LLM computations

Likelihood $\mathbb{P}_\wedge$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 and ... and Pn:  
4   return a  
  
return a ⇔ P1 ∧ ... ∧ Pn
```

Modified $\mathbb{P}_\vee$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 or ... or Pn:  
4   return a  
  
return a ⇔ P1 ∨ ... ∨ Pn
```

- Since $(P_1 \wedge \dots \wedge P_n) \models (P_1 \vee \dots \vee P_n)$, monotonicity of WMC gives

$$\underbrace{\mathbb{P}_{\mathcal{M}}(\mathbf{T}^a \mid Q)}_{\text{likelihood}} = \mathbb{P}_\wedge(\text{return } a; \theta) \leq \mathbb{P}_\vee(\text{return } a; \theta)$$

i.e., the disjunction is an **upper bound** on the likelihood.

Going from programs to LLM computations

Likelihood $\mathbb{P}_{\wedge}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 and ... and Pn:  
4   return a
```

Modified $\mathbb{P}_{\vee_k}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
3 ## at least k hold  
4 assert sum(P1, ..., Pn) ≥ k  
5 return a
```

Going from programs to LLM computations

Likelihood $\mathbb{P}_{\wedge}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 and ... and Pn:  
4   return a
```

Modified $\mathbb{P}_{\vee_k}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
3 ## at least k hold  
4 assert sum(P1, ..., Pn) ≥ k  
5 return a
```

► Let $U_k := \mathbb{P}_{\vee_k}(\text{at least } k \text{ of } n \text{ hold})$; then

$$\underbrace{U_1}_{\vee} \geq U_2 \geq \dots \geq \underbrace{U_n}_{\wedge} = \mathbb{P}_{\mathcal{M}}(\mathbf{T}^a \mid Q)$$

a hierarchy of upper bounds tightening to the likelihood.

Going from programs to LLM computations

Likelihood $\mathbb{P}_\wedge$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
  
3 if P1 and ... and Pn:  
4   return a
```

Modified $\mathbb{P}_{\vee_k}$

```
1 P1 =~ Flip(s1, θ1) ...  
2 Pn =~ Flip(sn, θn)  
3 k =~ Categorical(φ)  
4 assert sum(P1, ..., Pn) ≥ k  
5 return a
```

Making the threshold $k \sim \text{Categorical}(\varphi)$ a **random variable** makes it **learnable** — learning φ is just inference in the *same* program.

Going from programs to LLM computations

```
Likelihood  $P_{\Lambda}$   
1  $P_1 \sim \text{Flip}(s_1, \theta_1) \dots$   
2  $P_n \sim \text{Flip}(s_n, \theta_n)$   
  
3 if  $P_1$  and  $\dots$  and  $P_n$ :  
    return
```

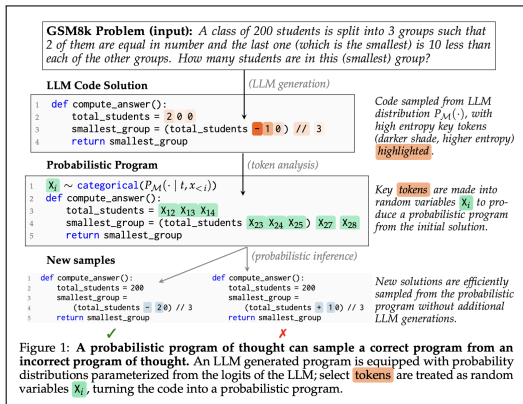
We can learn lots about LLMs from simple PPs,
PP inference is also often cheaper

Going from programs to LLM computations

```
Likelihood  $P_{\Lambda}$   
1  $P_1 \sim \text{Flip}(s_1, \theta_1) \dots$   
2  $P_n \sim \text{Flip}(s_n, \theta_n)$   
  
3 if  $P_1$  and  $\dots$  and  $P_n$ :  
4     return 2
```

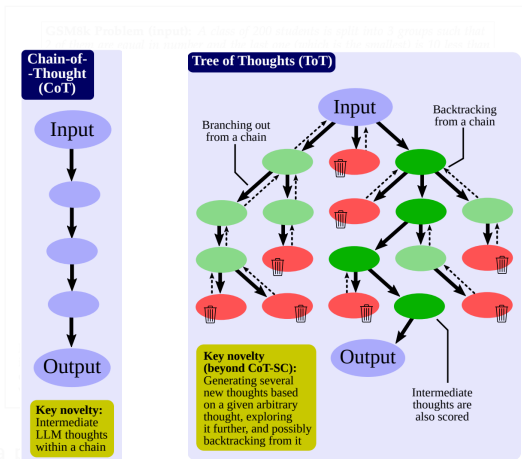
How much reasoning can we do using just a single forward call?

Probabilistic Programs of Thought (Garg et al.2026)



- Define a probabilistic program over LLM logits, use to cheaply obtain new samples, as a proposal distribution for importance sampling.

Probabilistic Programs of Thought (Garg et al. 2026)



from Besta et al. (2023) *Graph of Thoughts AAI*

Multi-answer chain-of-thought

- ▶ **Idea:** Generate multiple chains-of-thought in a single forward pass, use likelihood, other direct feedback to find the best answer.

Multi-answer chain-of-thought

- **Idea:** Generate multiple chains-of-thought in a single forward pass, use likelihood, other direct feedback to find the best answer.

Instruction: Please answer each question thinking step-by-step. **Rather than giving a single answer, however, I want you to express different possible reasoning paths.** Number your **steps** and identify **step dependencies**.

Query: Would a sea kayak fit into a normal car?

Example Trace (*Llama-3.3-70B-turbo-instruct*) **1.** A sea kayak is typically around 14-18 feet long **2.** Most cars have a maximum interior length of around 6-8 feet **3.** Based on 1 and 2, the answer is: no (ref. 1, 2) (final answer: no) **4.** Some cars have roof racks that can carry long items like kayaks **5.** Alternatively, based on 4, the answer is: yes (ref. 4) (final answer: yes)

Step likelihoods $\theta_{\text{likelihood}}$ (length normalized) (*for no*) **P1** 0.839 **P2** 0.99 **P3** 0.951. (*for yes*) **P4** 0.921 **P5** 0.92

Program sketch: if **P1** and **P2** and **P3**: return "no"
elif **P4** and **P5**: return "yes"

Multi-answer chain-of-thought

- **Idea:** Generate multiple chains-of-thought in a single forward pass, use likelihood, other direct feedback to find the best answer.

Instruction: Please answer each question thinking step-by-step. **Rather than giving a single answer, however, I want you to express different possible reasoning paths.** Number your **steps** and identify **step dependencies**.

Query: Would a sea kayak fit into a normal car?

Example Trace (*Llama-3.3-70B-turbo-instruct*) **1.** A sea kayak is typically around 14-18 feet long **2.** Most cars have a maximum interior length of around 6-8 feet **3.** Based on 1 and 2, the answer is: no (ref. 1, 2) (final answer: no) **4.** Some cars have roof racks that can carry long items like kayaks **5.** Alternatively, based on 4, the answer is: yes (ref. 4) (final answer: yes)

Step likelihoods $\theta_{\text{likelihood}}$ (length normalized) (*for no*) **P1** 0.839 **P2** 0.99 **P3** 0.951. (*for yes*) **P4** 0.921 **P5** 0.92

Program sketch: `if P1 and P2 and P3: return "no"`
`elif P4 and P5: return "yes"`

Problem: branching likelihoods programs are confounded by length, mean support is rarely matched by raw likelihood.

Multi-answer chain-of-thought

- **Idea:** Generate multiple chains-of-thought in a single forward pass, use likelihood, other direct feedback to find the best answer.

Instruction: Please answer each question thinking step-by-step. **Rather than giving a single answer, however, I want you to express different possible reasoning paths.** Number your **steps** and identify **step dependencies**.

Query: Would a sea kayak fit into a normal car?

Example Trace (*Llama-3.3-70B-turbo-instruct*) **1.** A sea kayak is typically around 14-18 feet long **2.** Most cars have a maximum interior length of around 6-8 feet **3.** Based on 1 and 2, the answer is: no (ref. 1, 2) (final answer: no) **4.** Some cars have roof racks that can carry long items like kayaks **5.** Alternatively, based on 4, the answer is: yes (ref. 4) (final answer: yes)

Step likelihoods $\theta_{\text{likelihood}}$ (length normalized) (*for no*) **P1** 0.839 **P2** 0.99 **P3** 0.951. (*for yes*) **P4** 0.921 **P5** 0.92

Program sketch: `if P1 and P2 and P3: return "no"`
`elif P4 and P5: return "yes"`

- Introduced a new construct for aggregation scores using more expressive differentiable operators, $[0, 1]^n \rightarrow [0, 1]$

F: `bool` = `Factor`("aggr", $f(P1, P2, P3)$)

Multi-answer chain-of-thought

- **Idea:** Generate multiple chains-of-thought in a single forward pass, use likelihood, other direct feedback to find the best answer.

Instruction: Please answer each question thinking step-by-step. Rather than giving a single answer, however, I want you to express different possible reasoning paths. Number your steps and identify step dependencies.

Query: Would a sea kayak fit into a normal car?

Example Trace (Llama-3.3-70B-turbo-instruct) 1. A sea kayak is typically around 14-18 feet long 2. Most cars have a maximum interior length of around 6-8 feet 3. have roof racks that can carry long items like kayaks 3. Alternatively, based on 4, the answer is: yes (ref. 4) {final answer: yes}

Step likelihoods $\theta_{\text{likelihood}}$ (length normalized) (for no) P1 0.839 P2 0.99 P3 0.951 (for yes) P4 0.921 P5 0.92

Program sketch: if P1 and P2 and P3: return "no"
elif P4 and P5: return "yes"

Deriving new prompting strategies from first principles

(A) Multi-answer Chain-of-thought

Query Q: Would a sea kayak fit into a normal car? Think step-by-step

LLM CoT Reasoning Trace (single forward call)

For yes (LLM verbal confidence: 0.4)

A sea kayak can vary in size, but some fit into larger vehicles. (0.642) ✓

Some cars, especially SUVs, have foldable seats and ample space. (0.595) ✓

If the kayak is small and the car large enough, it might fit. (0.698) ✓

Based on this, the answer is: **yes (answer)** ✓

For no (LLM verbal confidence: 0.7)

Sea kayaks are typically 12–18 feet long. (0.831) ✗

Most cars cannot handle such long objects. (0.733) ✗

Even with foldable seats, a kayak exceeds interior space. (0.676) ✗

Based on this, the answer is: **no (answer)** ✗

(B) Copper Program

```

yes: bool ~ Flip("yes", 0.4)
no: bool ~ Flip("no", 0.7)
A: bool ~ Categorical("yes", Combine(yes, no))

P1: bool ~ Flip("s1", 0.642)
P2: bool ~ Flip("s2", 0.595)
P3: bool ~ Flip("s3", 0.698)
EY: bool ~ Factor("yes_E", LogitPool(P1, P2, P3))

P5: bool ~ Flip("s5", 0.831)
P6: bool ~ Flip("s6", 0.733)
P7: bool ~ Flip("s7", 0.676)
EN: bool ~ Factor("no_E", LogitPool(P5, P6, P7))
observe((A and EY) or ((not A) and EN))
if A: return "yes"
else: return "no"
    
```

(C) Probabilistic model

$\theta_{\text{verbal}} = (0.4, 0.7)$

$P_{\text{prior}}(A | Q; \theta_{\text{verbal}})$

$\theta_{\text{likelihood}} = (0.642, 0.595, 0.698, 0.831, 0.733, 0.676)$

$P_{\text{evidence}}(\mathbf{T} | A, Q; \theta_{\text{lik.}})$

$P_{\text{posterior}}(A | \mathbf{T}, Q; \theta_{\text{verbal, lik.}})$

- **Bayesian multi-answer CoT**: inverts the structure of CoT ($a, s_1, \dots, s_n$), aggregates step scores, combines LLM likelihood with verbal feedback.

Deriving new prompting strategies from first principles

(A) Multi-answer Chain-of-thought

Query Q: Would a sea kayak fit into a normal car? Think step-by-step

LLM CoT Reasoning Trace (single forward call)

For yes (LLM verbal confidence: 0.4)

A sea kayak can vary in size, but some fit into larger vehicles. (0.642) ✓

Some cars, especially SUVs, have foldable seats and ample space. (0.595) ✓

If the kayak is small and the car large enough, it might fit. (0.698) ✓

Based on this, the answer is: **yes (answer)** ✓

For no (LLM verbal confidence: 0.7)

Sea kayaks are typically 12–18 feet long. (0.831) ✗

Most cars cannot handle such long objects. (0.733) ✗

Even with foldable seats, a kayak exceeds interior space. (0.676) ✗

Based on this, the answer is: **no (answer)** ✗

(B) Copper Program

```

yes: bool ~ Flip("yes", 0.4)
no: bool ~ Flip("no", 0.7)
A: bool ~ Categorical("yes", Combine(yes, no))

P1: bool ~ Flip("s1", 0.642)
P2: bool ~ Flip("s2", 0.595)
P3: bool ~ Flip("s3", 0.698)
EY: bool ~ Factor("yes_E", LogitPool(P1, P2, P3))

P5: bool ~ Flip("s5", 0.831)
P6: bool ~ Flip("s6", 0.733)
P7: bool ~ Flip("s7", 0.676)
EN: bool ~ Factor("no_E", LogitPool(P5, P6, P7))
observe((A and EY) or ((not A) and EN))
if A: return "yes"
else: return "no"
    
```

(C) Probabilistic model

$\theta_{\text{verbal}} = (0.4, 0.7)$

$P_{\text{prior}}(A | Q; \theta_{\text{verbal}})$

$\theta_{\text{likelihood}} = (0.642, 0.595, 0.698, 0.831, 0.733, 0.676)$

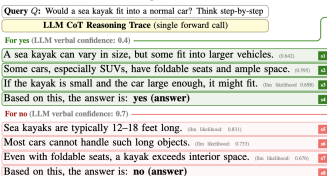
$P_{\text{evidence}}(\mathbf{T} | A, Q; \theta_{\text{lik.}})$

$P_{\text{posterior}}(A | \mathbf{T}, Q; \theta_{\text{verbal, lik.}})$

- **Bayesian multi-answer CoT**: inverts the structure of CoT ($a, s_1, \dots, s_n$), aggregates step scores, combines LLM likelihood with verbal feedback.

Deriving new prompting strategies from first principles

(A) Multi-answer Chain-of-thought



(B) Copper Program

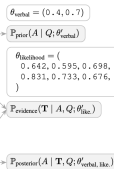
```

yes: bool ~ Flip("yes", 0.4)
no: bool ~ Flip("no", 0.7)
A: bool ~ Categorical("yes", Combine(yes, no))

P1: bool ~ Flip("s1", 0.642)
P2: bool ~ Flip("s2", 0.595)
P3: bool ~ Flip("s3", 0.698)
EY: bool ~ Factor("yes_E", LogitPool(P1, P2, P3))

P5: bool ~ Flip("s5", 0.831)
P6: bool ~ Flip("s6", 0.733)
P7: bool ~ Flip("s7", 0.676)
EN: bool ~ Factor("no_E", LogitPool(P5, P6, P7))
observe((A and EY) or ((not A) and EN))
if A: return "yes"
else: return "no"
    
```

(C) Probabilistic model

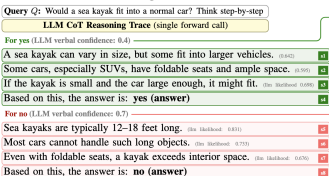


- **Bayesian multi-answer CoT**: inverts the structure of CoT ($a, s_1, \dots, s_n$), aggregates step scores, combines LLM likelihood with verbal feedback.

Looked at program robustness, robustness of verbal vs. likelihood feedback, the reusability of CoTs across models.

Deriving new prompting strategies from first principles

(A) Multi-answer Chain-of-thought



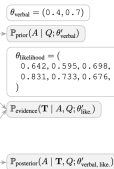
(B) Copper Program

```
yes: bool ~ Flip("yes", 0.4)
no: bool ~ Flip("no", 0.7)
A: bool ~ Categorical("yes", Combine(yes, no))

P1: bool ~ Flip("s1", 0.642)
P2: bool ~ Flip("s2", 0.595)
P3: bool ~ Flip("s3", 0.698)
EY: bool ~ Factor("yes_E", LogitPool(P1, P2, P3))

P5: bool ~ Flip("s5", 0.831)
P6: bool ~ Flip("s6", 0.733)
P7: bool ~ Flip("s7", 0.676)
EN: bool ~ Factor("no_E", LogitPool(P5, P6, P7))
observe((A and EY) or ((not A) and EN))
if A: return "yes"
else: return "no"
```

(C) Probabilistic model



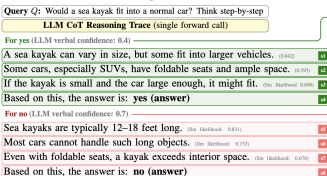
- **Bayesian multi-answer CoT**: inverts the structure of CoT ($a, s_1, \dots, s_n$), aggregates step scores, combines LLM likelihood with verbal feedback.

Findings: models reason robustly over CoTs from *much weaker* models.

Task	Solves it alone		Reasons over
	Qwen (<i>small</i>)	Llama (<i>large</i>)	Qwen's CoTs (Llama)
BoolQ	69.5	90.8	90.6
StrategyQA	45.7	67.6	68.0

Deriving new prompting strategies from first principles

(A) Multi-answer Chain-of-thought



(B) Copper Program

```

yes: bool ~ Flip("yes", 0.4)
no: bool ~ Flip("no", 0.7)
A: bool ~ Categorical("yes", Combine(yes, no))

P1: bool ~ Flip("s1", 0.642)
P2: bool ~ Flip("s2", 0.595)
P3: bool ~ Flip("s3", 0.698)
EY: bool ~ Factor("yes_E", LogitPool(P1, P2, P3))

P5: bool ~ Flip("s5", 0.831)
P6: bool ~ Flip("s6", 0.733)
P7: bool ~ Flip("s7", 0.676)
EN: bool ~ Factor("no_E", LogitPool(P5, P6, P7))
observe((A and EY) or ((not A) and EN))
if A: return "yes"
else: return "no"

```

(C) Probabilistic model

$\theta_{\text{verbal}} = (0.4, 0.7)$

$P_{\text{prior}}(A | Q; \theta_{\text{verbal}})$

$\theta_{\text{likelihood}} = (0.642, 0.595, 0.698, 0.831, 0.733, 0.676)$

$P_{\text{evidence}}(T | A, Q; \theta_{\text{lik}})$

$P_{\text{posterior}}(A | T, Q; \theta_{\text{verbal, lik}})$

- **Bayesian multi-answer CoT**: inverts the structure of CoT ($a, s_1, \dots, s_n$), aggregates step scores, combines LLM likelihood with verbal feedback.

Findings: *verbalized* probabilities are as robust as (or better than) raw logits.

Model	Pure Verbalized	Logit step + verb prior	Verb step + logit prior	Pure Logits
GPT-4o	78.19	77.97	73.40	73.40
Qwen-2.5-7B	65.03	65.38	50.88	51.16
Llama-3.3-70B	76.71	76.42	70.58	69.95

Deriving new prompting strategies from first principles

(A) Multi-answer Chain-of-thought

Query Q: Would a van knock it into a second car? Think step-by-step.
LLM: Answering True (single forward call)

The van is 12-18 feet long, 6-8 feet wide, and 6-8 feet high. It is a large vehicle.
A van knock can vary in size, but some fit into larger vehicles.
Some cars, especially SUVs, have foldable seats and ample space.
If the knock is small and the car large enough, it might fit.
Based on this, the answer is: yes (answer)

The van is 12-18 feet long, 6-8 feet wide, and 6-8 feet high. It is a large vehicle.
A van knock can vary in size, but some fit into larger vehicles.
Some cars, especially SUVs, have foldable seats and ample space.
If the knock is small and the car large enough, it might fit.
Based on this, the answer is: yes (answer)

(B) Grepper Program

```
yes: bool := P[Knock] * P[Car] * P[Fit]
no: bool := P[Knock] * P[Car] * P[NotFit]
fit: bool := Grepper[Knock, Car, P[Fit], P[NotFit]]

fit: bool := P[Knock] * P[Car] * P[Fit]
no: bool := P[Knock] * P[Car] * P[NotFit]
fit: bool := P[Knock] * P[Car] * P[Fit]
no: bool := P[Knock] * P[Car] * P[NotFit]

fit: bool := P[Knock] * P[Car] * P[Fit]
no: bool := P[Knock] * P[Car] * P[NotFit]
fit: bool := P[Knock] * P[Car] * P[Fit]
no: bool := P[Knock] * P[Car] * P[NotFit]
```

(C) Probabilistic model

$P_{\text{Knock}} = (0.5, 0.5, 0.5)$
 $P_{\text{Car}} = (0.5, 0.5, 0.5)$
 $P_{\text{Fit}} = (0.5, 0.5, 0.5)$
 $P_{\text{NotFit}} = (0.5, 0.5, 0.5)$
 $P_{\text{Knock}} * P_{\text{Car}} * P_{\text{Fit}} = 0.125$
 $P_{\text{Knock}} * P_{\text{Car}} * P_{\text{NotFit}} = 0.125$
 $P_{\text{Knock}} * P_{\text{Car}} * P_{\text{Fit}} = 0.125$
 $P_{\text{Knock}} * P_{\text{Car}} * P_{\text{NotFit}} = 0.125$

PP offers many new empirical directions to explore
aggregates step scores, combines LLM likelihood with verbal feedback.

Conclusions and questions

- ▶ Proposed viewing much of LLM development as a form of high level programming, **language model programming**, helpful for semantics.
- ▶ Discussed a particular example language for prompting and chain-of-thought, looking forward:

Conclusions and questions

- ▶ Proposed viewing much of LLM development as a form of high level programming, **language model programming**, helpful for semantics.
- ▶ Discussed a particular example language for prompting and chain-of-thought, looking forward:
 - ▶ Lots of room for making the language more expressive (e.g., *iteration, numbers, functions, ...*), extending to more complex reasoning phenomena (e.g., *agents*), constructs for formal verification, ...
- ▶ **Important point:** The issue in LLM modeling is understanding what the computations mean, not how to do them.

Questions



Nice connection between CoT and Probabilistic Programming, but the tractability and circuits are not so crucial for that

Official Review by Reviewer VtKw 📅 02 Jul 2026, 16:22 (modified: 13 Aug 2026, 18:52) 👁 Everyone

📄 Revisions

Questions



Nice connection between CoT and Probabilistic Programming, but the tractability and circuits are not so crucial for that

Official Review by Reviewer VtKw 📅 02 Jul 2026, 16:22 (modified: 13 Aug 2026, 18:52) 👁 Everyone

📄 Revisions

Questions:

- ▶ Is all of this indeed irrelevant for the TPM community?
- ▶ Can we characterize the circuit / TPM classes relevant to LLM computation?
- ▶ Should we care about these kinds of special cases?

Thank you.

References I

- Beurer-Kellner, L., Fischer, M., and Vechev, M. (2023). Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.
- Bogin, B., Yang, K., Gupta, S., Richardson, K., Bransom, E., Clark, P., Sabharwal, A., and Khot, T. (2024). Super: Evaluating agents on setting up and executing tasks from research repositories. *Proceedings of EMNLP*.
- Bragg, J., D’Arcy, M., Balepur, N., Bareket, D., Dalvi, B., Feldman, S., Haddad, D., Hwang, J. D., Jansen, P., Kishore, V., Majumder, B. P., Naik, A., Rahamimov, S., Richardson, K., Singh, A., Surana, H., Tiktinsky, A., Vasu, R., Wiener, G., Anastasiades, C., Candra, S., Dunkelberger, J., Emery, D., Evans, R., Hamada, M., Huff, R., Kinney, R., Latzke, M., Lochner, J., Lozano-Aguilera, R., Nguyen, C., Rao, S., Tanaka, A., Vlahos, B., Clark, P., Downey, D., Goldberg, Y., Sabharwal, A., and Weld, D. S. (2026). Astabench: Rigorous benchmarking of ai agents with a scientific research suite. *Proceedings of ICLR*.
- Chen, J., Yuan, S., Ye, R., Majumder, B. P., and Richardson, K. (2023). Put your money where your mouth is: Evaluating strategic planning and execution of llm agents in an auction arena. *arXiv preprint arXiv:2310.05746*.
- Cheng, J., Clark, P., and Richardson, K. (2025). Language modeling by language models. *Proceedings of Neurips*.
- Cheng, J., Richardson, K., and Chin, P. (2026). Analytica: Soft propositional reasoning for robust and scalable llm-driven analysis. *Proceedings of ICLR*.

References II

- De Raedt, L. and Kimmig, A. (2015). Probabilistic (logic) programming concepts. *Machine Learning*, 100:5–47.
- De Raedt, L., Kimmig, A., and Toivonen, H. (2007). Problog: A probabilistic prolog and its application in link discovery. In *IJCAI 2007, Proceedings of the 20th international joint conference on artificial intelligence*, pages 2462–2467. IJCAI-INT JOINT CONF ARTIF INTELL.
- Dohan, D., Xu, W., Lewkowycz, A., Austin, J., Bieber, D., Lopes, R. G., Wu, Y., Michalewski, H., Saurous, R. A., Sohl-Dickstein, J., et al. (2022). Language model cascades. *arXiv preprint arXiv:2207.10342*.
- Dries, A., Kimmig, A., Meert, W., Renkens, J., Van den Broeck, G., Vlasselaer, J., and De Raedt, L. (2015). Problog2: Probabilistic logic programming. In *Joint european conference on machine learning and knowledge discovery in databases*, pages 312–315. Springer.
- Fierens, D., Van den Broeck, G., Renkens, J., Shterionov, D., Gutmann, B., Thon, I., Janssens, G., and De Raedt, L. (2015). Inference and learning in probabilistic logic programs using weighted boolean formulas. *Theory and Practice of Logic Programming*, 15(3):358–401.
- Garg, P., Geh, R. L., Israel, D., Millstein, T., Richardson, K., and Broeck, G. V. d. (2026). Probabilistic programs of thought. *arXiv preprint arXiv:2604.17290*.

References III

- Huang, X., Bakalova, A., Bhattamishra, S., Merrill, W., and Hahn, M. (2026). Discovering interpretable algorithms by decompiling transformers to rasp. *arXiv preprint arXiv:2602.08857*.
- Mandel, L., Baudart, G., Vaziri, M., and Hirzel, M. (2026). Ppdl: Llm-based flows as probabilistic programs. In *Forty-third International Conference on Machine Learning*.
- Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., and De Raedt, L. (2018). Deepproblog: Neural probabilistic logic programming. *Advances in neural information processing systems*, 31.
- Richardson, K., Anderson, C., Balakrishnan, P., Ban, T., Ladia, D., Gupta, A., and Hudspeth, M. (2026a). From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models. *Proceedings of EMNLP*.
- Richardson, K., Feng, Y., Garg, P., Cheng, J., and Roth, D. (2026b). CoTs as Tractable Probabilistic Programs. *Proceedings of TPM*.
- Richardson, K., Srikumar, V., and Sabharwal, A. (2025). Understanding the logic of direct preference alignment through logic. *Proceedings of ICML*.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- Weiss, G., Goldberg, Y., and Yahav, E. (2021). Thinking like transformers. In *International Conference on Machine Learning*, pages 11080–11090. PMLR.

References IV

- Wu, C., Yang, S., and Murali, A. (2026). Imprompt: A language framework for prompt programming. *arXiv preprint arXiv:2607.22683*.
- Yang, A. and Chiang, D. (2024). Counting like transformers: Compiling temporal counting logic into softmax transformers. *arXiv preprint arXiv:2404.04393*.
- Yang, A., Chiang, D., and Angluin, D. (2024). Masked hard-attention transformers recognize exactly the star-free languages. *Advances in Neural Information Processing Systems*, 37:10202–10235.
- Yang, R., Chen, J., Zhang, Y., Yuan, S., Chen, A., Richardson, K., Xiao, Y., and Yang, D. (2025). Selfgoal: Your language agents already know how to achieve high-level goals. *Proceedings of NAACL*.
- Yu, H., Xuan, K., Li, F., Zhu, K., Lei, Z., Zhang, J., Qi, Z., Richardson, K., and You, J. (2025). Tinsyscientist: An interactive, extensible, and controllable framework for building research agents. *Proceedings of EMNLP*.
- Yu, H., You, J., Clark, P., Majumder, B. P., and Richardson, K. (2026). Artifactlinker: Linking scientific artifacts for automatic state-of-the-art discovery. *Proceedings of COLM*.
- Zhang, Y., Yuan, S., Hu, C., Richardson, K., Xiao, Y., and Chen, J. (2024). Timearena: Shaping efficient multitasking language agents in a time-aware simulation. *Proceedings of ACL*.